LECTURE · HMM PRINCIPLES

Hidden Markov Models

Principles of Evaluation, Decoding and Learning

Markov model → HMM parameters → Forward/Backward → Viterbi → Baum-Welch

Figures are adapted from the original lecture slides, with typos corrected.
One running example: Dr. Jung's daily activity and the hidden weather.

An HMM reads a hidden sequence through a visible one

Three ideas carry the whole lecture

1

Hidden states follow a Markov chain

The weather (rain/sun) changes day to day with transition matrix A; we never observe it directly.

2

Each hidden state emits an observation

Dr. Jung's activity (walk/shop/research) is drawn from emission matrix B of that day's weather.

3

Three problems, three dynamic programs

Evaluation → Forward/Backward, Decoding → Viterbi, Learning → Baum-Welch (EM).

λ = {A, B, π} fully specifies the model — everything else is computing with it.

Sequential data carries an order that matters

Background: the index changes from feature i to time t

Definition

- Ordinary data: features x1...xd
- Sequential data: x1...xT indexed by time
- Order of values is information

$$\mathbf{X} = (x_1, \dots x_i, x_{i+1}, \dots x_d)$$



$$\mathbf{X} = (x_1, \dots x_t, x_{t+1}, \dots x_T)$$

→ Sequential Data

Equipment routes in manufacturing form discrete sequences

Each row: equipment used at processes 1–10, then a normal/defect class

Read the table

- Rows = products
- Columns = process steps in time
- Cells = equipment ID
- Last column = quality class

Equipment sequential data in manufacturing processes

Time

	Proc 1	Proc 2	Proc 3	Proc 4	Proc 5	Proc 6	Proc 7	Proc 8	Proc 9	Proc 10	Class
1	Equip 1	Equip 2	Equip 1	Equip 1	Equip 2	Equip 1	Equip 4	Equip 2	Equip 2	Equip 1	Normal
2	Equip 2	Equip 4	Equip 2	Equip 2	Equip 4	Equip 2	Equip 4	Equip 2	Equip 4	Equip 2	Normal
3	Equip 1	Equip 3	Equip 1	Equip 1	Equip 3	Equip 1	Equip 3	Equip 1	Equip 3	Equip 1	Normal
...	...	...	...	...	...	...	...	...	...	...	...
2998	Equip 3	Equip 3	Equip 3	Equip 3	Equip 3	Equip 3	Equip 3	Equip 3	Equip 3	Equip 3	Defect
2999	Equip 4	Equip 1	Equip 2	Equip 4	Equip 6	Equip 5	Equip 5	Equip 1	Equip 2	Equip 3	Defect
3000	Equip 5	Equip 5	Equip 1	Equip 3	Equip 1	Equip 5	Equip 1	Equip 3	Equip 1	Equip 1	Defect

An HMM is a stochastic generative model for sequences

Hidden (not observed) + Markov model

Three words

- Hidden: states are not observed
- Markov: next state depends on current state
- Model: generates sequences with probabilities



01

PART 1 · MARKOV MODEL

The Markov Model

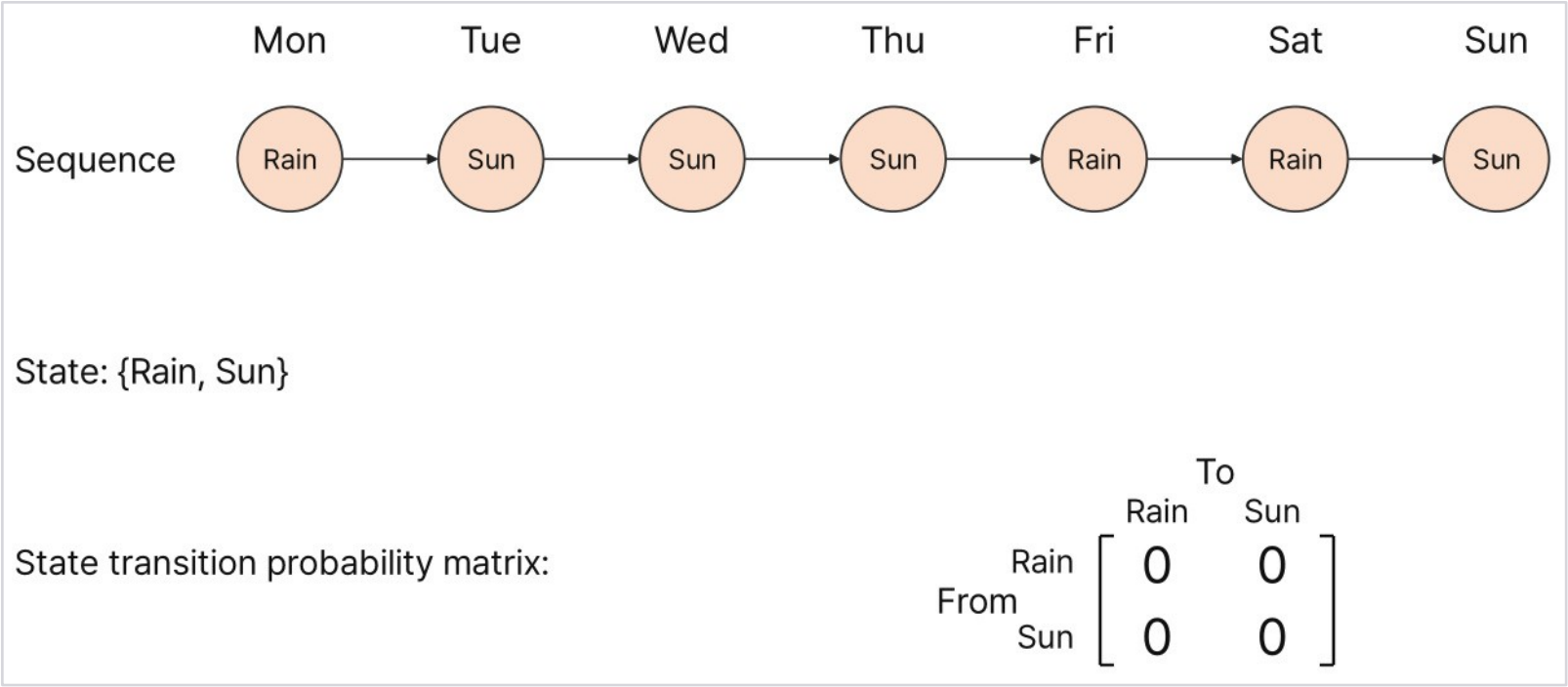
States, transitions and the Markov assumption

A Markov model summarises a sequence by transition probabilities

Example: weather over one week (rain or sun)

Set-up

- States: {rain, sun}
- Sequence: Mon → Sun
- Goal: fill the 2×2 matrix
- Rows = from, columns = to

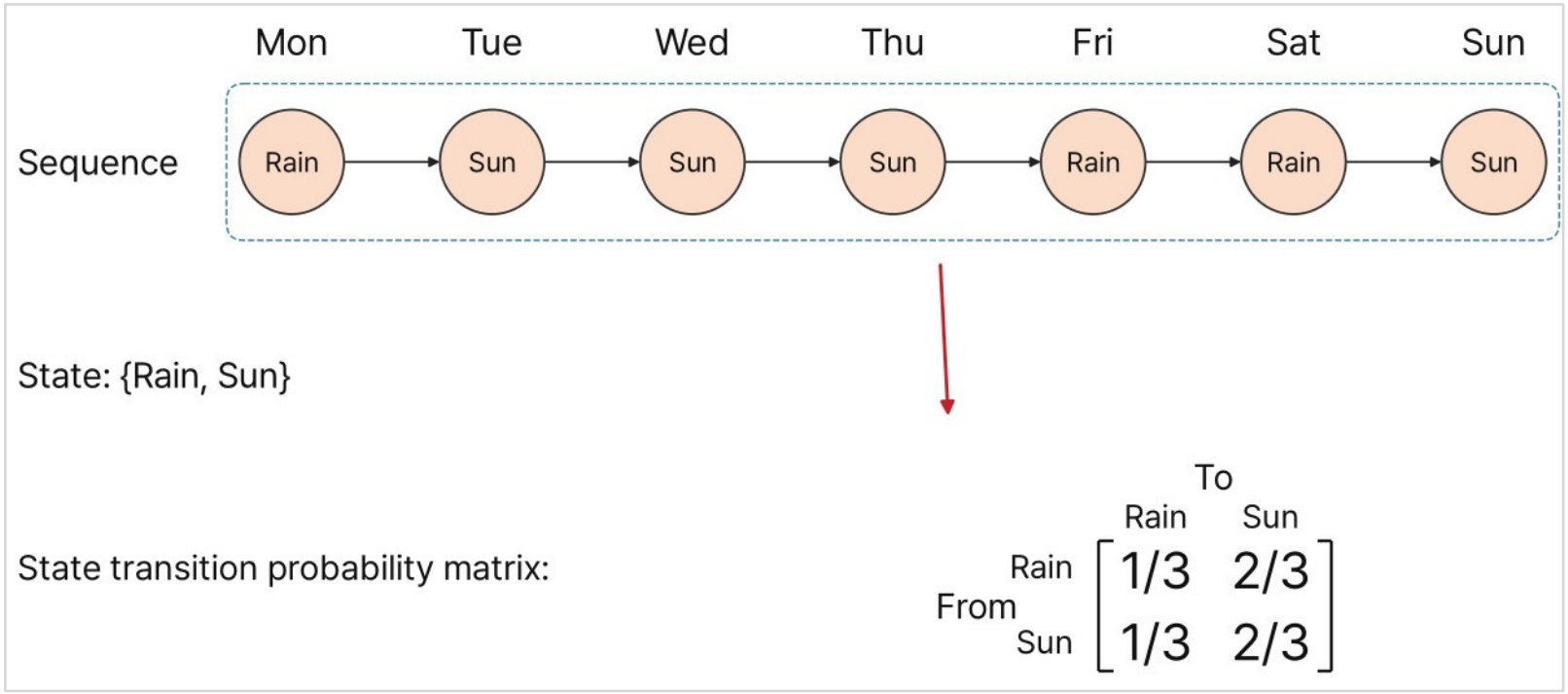


Transition probabilities are estimated by counting moves

Rain→rain 1 of 3 moves, sun→sun 2 of 3 moves

How to count

- From rain: 3 moves
- → rain 1, → sun 2
- From sun: 3 moves
- → rain 1, → sun 2
- Each row sums to 1



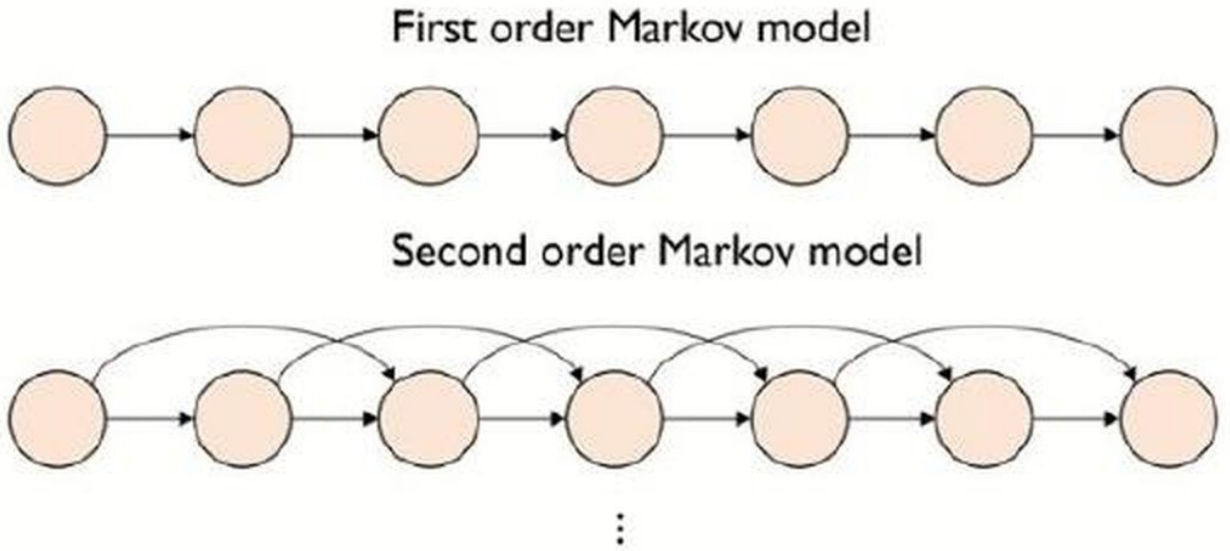
The Markov assumption cuts the dependence on long history

r-th order model: the state at t depends on the last r states only

Markov assumption

- 1st order: only the previous state
- 2nd order: previous two states
- Higher order = more arrows

- Markov assumption: the observation at time t depends only on the most recent r observations
- A transition from one state to another does not need the long history of earlier states
  - If r=1, $p(s_t|s_{t-1}s_{t-2}\cdots s_1) = p(s_t|s_{t-1})$
  - If r=2, $p(s_t|s_{t-1}s_{t-2}\cdots s_1) = p(s_t|s_{t-1}, s_{t-2})$
  - …



First-order and second-order Markov models differ only in memory

Formulas from the original slide

$$p(s_t \mid s_{t-1}s_{t-2}\cdots s_1) = p(s_t \mid s_{t-1}) \quad (r=1)$$

$$p(s_t \mid s_{t-1}s_{t-2}\cdots s_1) = p(s_t \mid s_{t-1}, s_{t-2}) \quad (r=2)$$

Why it matters

- The first-order model needs only an n×n transition matrix
- Most HMMs, including this lecture, use the first-order assumption

Matrix form and graph form describe the same Markov model

Entry a_ij = probability of moving from state i to state j

Two views

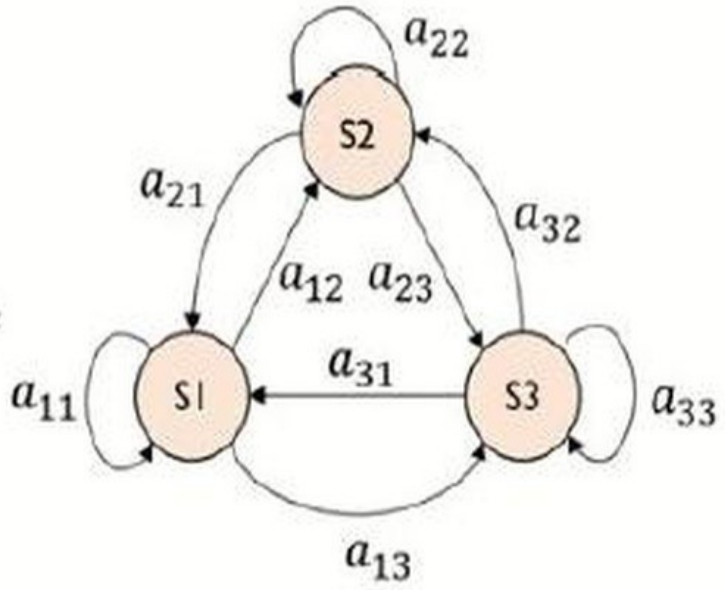
- Matrix: rows t−1, columns t
- Graph: arrows weighted by a_ij
- Self-loops = a_ii

❖ State transition probability matrix $A(a_{ij})$

Matrix Form

$$\begin{array}{cc} & T \\ & \begin{array}{ccc} S1 & S2 & S3 \end{array} \\ T-1 \begin{array}{c} S1 \\ S2 \\ S3 \end{array} & \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \end{array}$$

Graph Form



02

PART 2 · HMM CONCEPT

Hidden Markov Models

Two sequences linked in time

An HMM models two sequences that occur at the same time

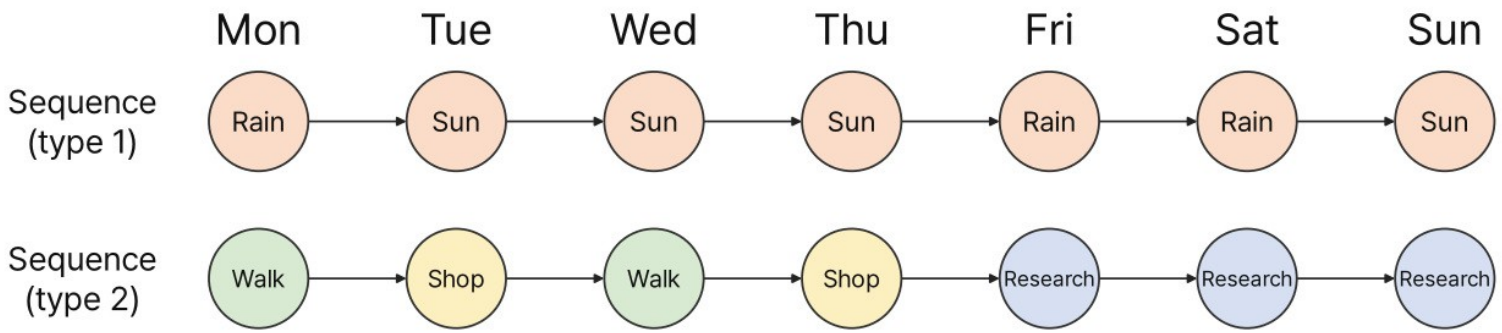
Type 1: weather (rain/sun) · Type 2: activity (walk, shop, research)

Two sequences

- Type 1: states s1… sT
- Type 2: observations o1…oT
- Model each and their link

◆ What is a Hidden Markov model?

- Type 1 state sequence: $(s_1, s_2, s_3, \ldots, s_{T-1}, s_T)$
- Type 2 state sequence: $(o_1, o_2, o_3, \ldots, o_{T-1}, o_T)$



Models two state sequences occurring at the same time, and the relationship between them

The weather is hidden while the activity is observable

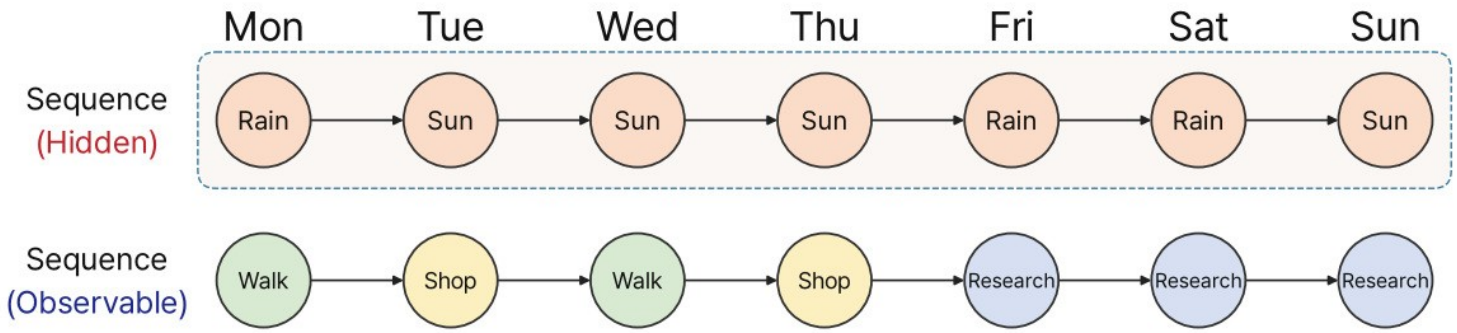
We see what Dr. Jung did, not what the weather was

Hidden vs observable

- Hidden: weather sequence
- Observable: activity sequence
- Inference runs upward

◆ What is a Hidden Markov model?

- Hidden state sequence: $(s_1, s_2, s_3, \ldots, s_{T-1}, s_T)$
- Observable state sequence: $(o_1, o_2, o_3, \ldots, o_{T-1}, o_T)$



The Type 1 sequence is hidden; the Type 2 sequence is observable

Observations depend on the hidden state of the same day

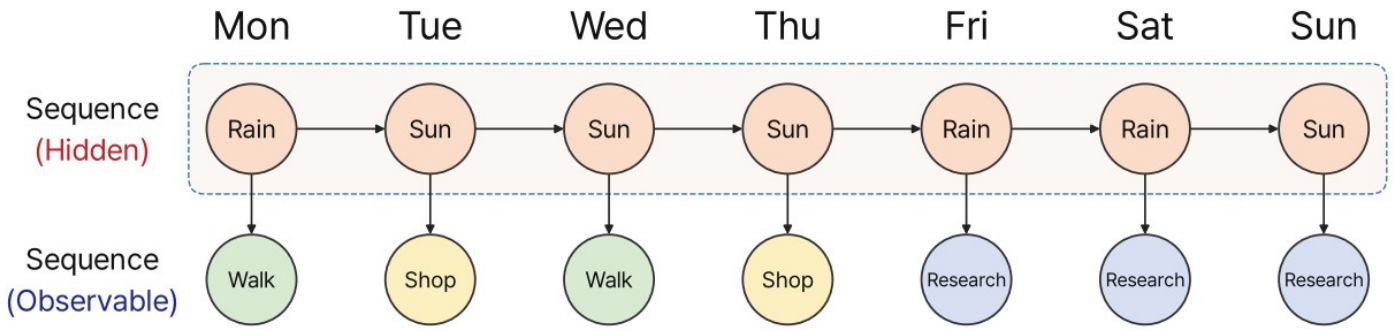
Hidden sequence obeys the Markov assumption; observations hang from it

Structure

- Hidden chain carries time order
- Each day emits one activity
- No arrows between activities

◆ What is a Hidden Markov model?

- Hidden state sequence: $(s_1, s_2, s_3, \ldots, s_{T-1}, s_T)$
- Observable state sequence: $(o_1, o_2, o_3, \ldots, o_{T-1}, o_T)$



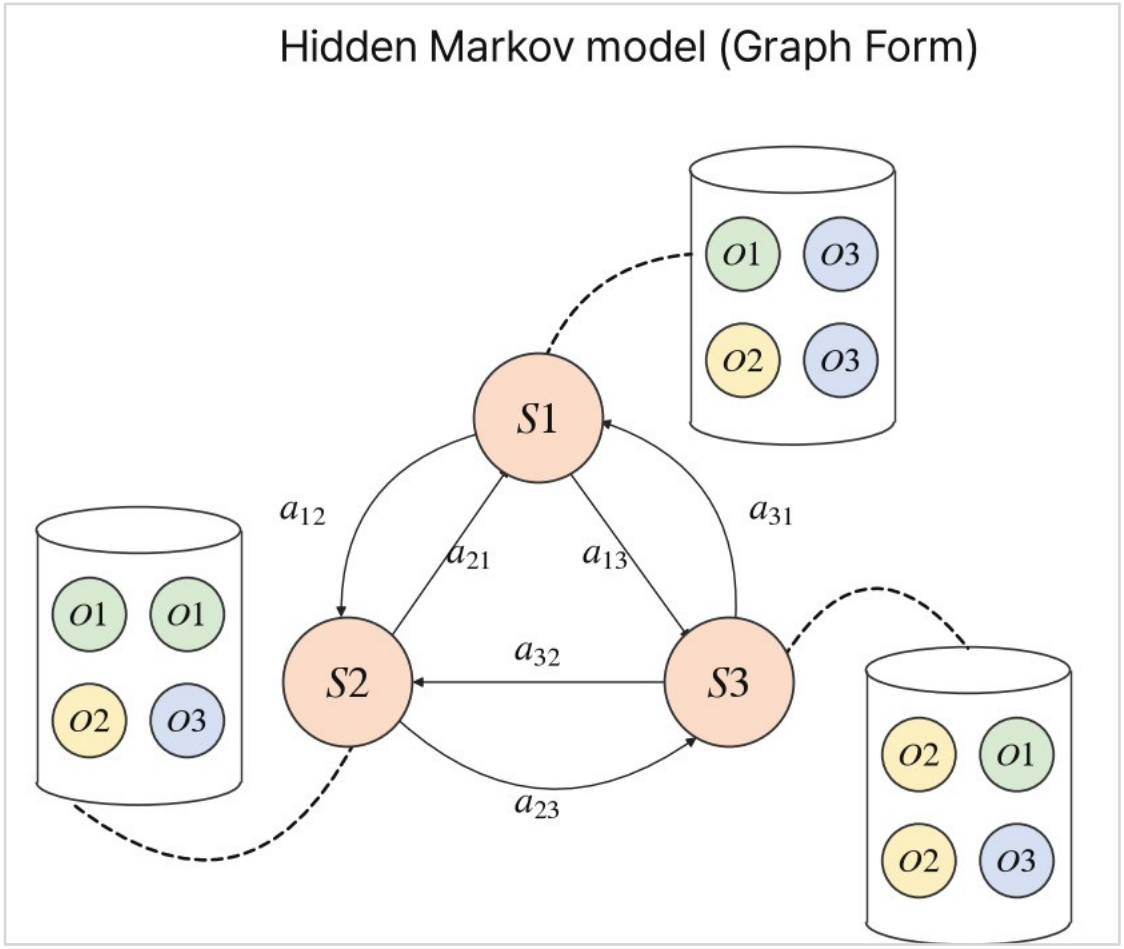
- Hidden sequence follows the Markov assumption → reflects sequential structure
- Observable sequence depends on the Hidden sequence, which carries the sequential structure

Each hidden state owns its own urn of observations

Graph form: states s1–s3 with transition arrows and observation urns o1–o3

Urn picture

- Visit a state
- Draw a ball from its urn
- Urn mix differs by state

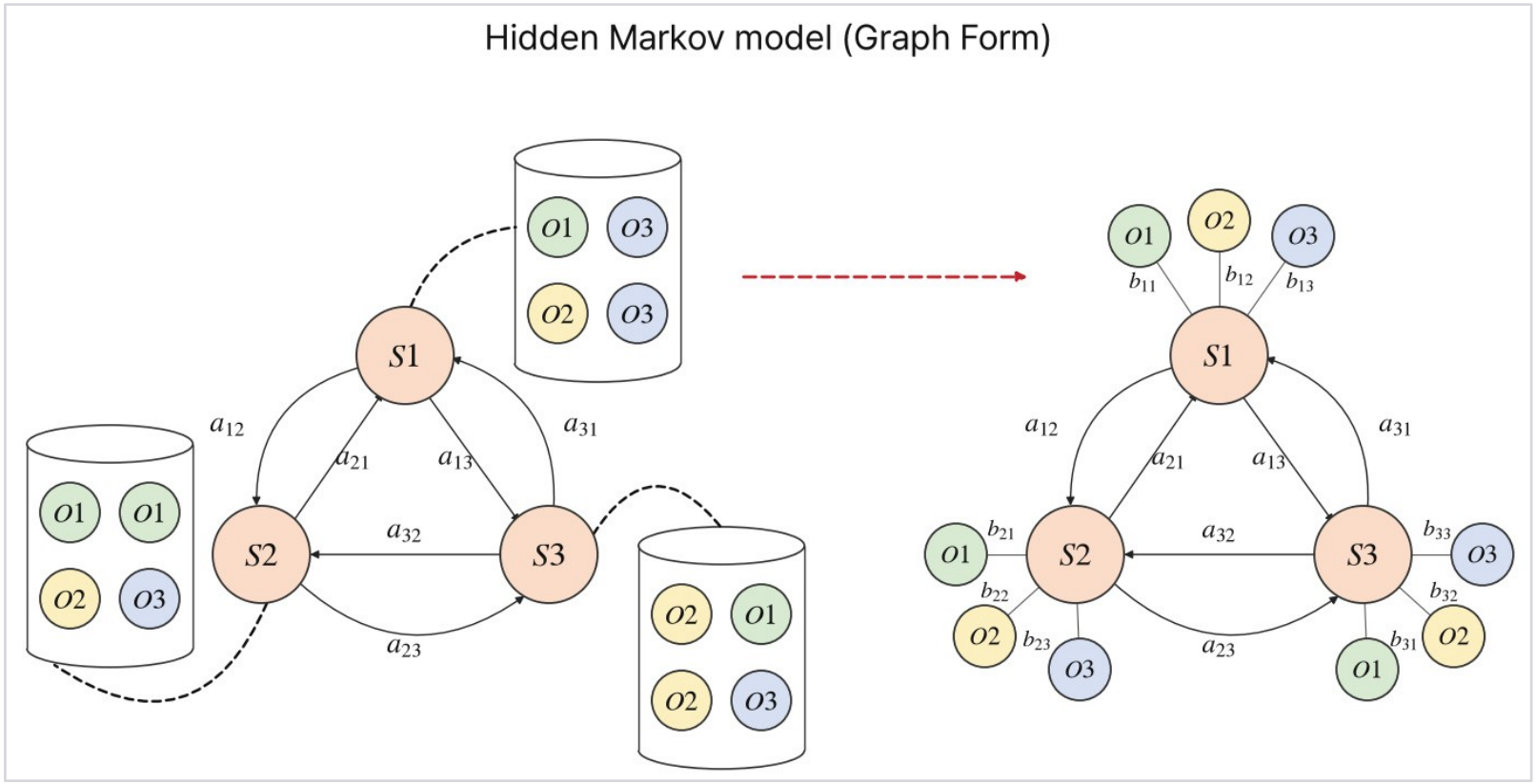


Urns become emission probabilities on the graph

Ball frequencies in each urn turn into weights b_jk

From urn to b

- Left: urn picture
- Right: emission edges
- b = share of each ball



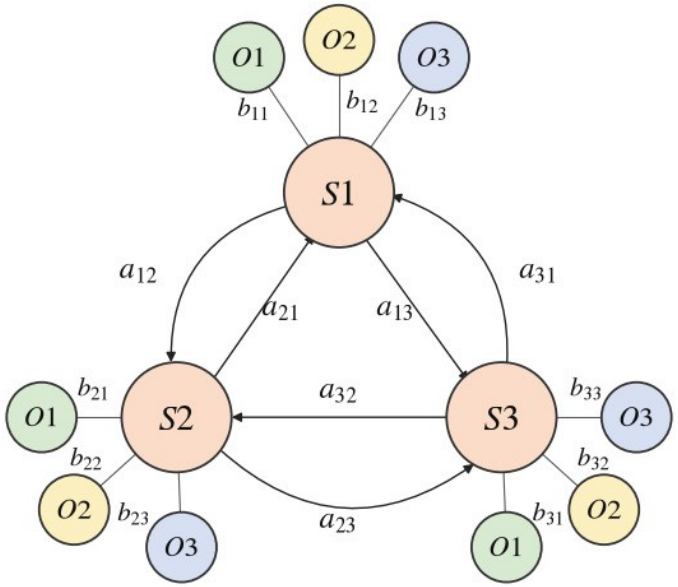
Transition matrix A and emission matrix B are the core parameters

A: state → state · B: state → observation

Two matrices

- A: n×n transitions
- B: emission per state
- Original typos a22, a32 corrected to b22, b32

- $A(a_{ij})$: State transition probability matrix
- $B(b_{jk})$: Emission probability matrix



$$A = \begin{array}{c} \\ S1 \\ S2 \\ S3 \end{array} \begin{array}{c} \begin{array}{ccc} S1 & S2 & S3 \end{array} \\ \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \end{array}$$

$$B = \begin{array}{c} \\ S1 \\ S2 \\ S3 \end{array} \begin{array}{c} \begin{array}{ccc} O1 & O2 & O3 \end{array} \\ \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} \end{array}$$

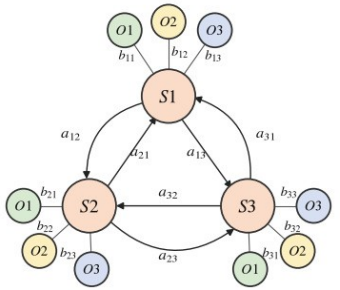
The initial distribution π decides where the chain starts

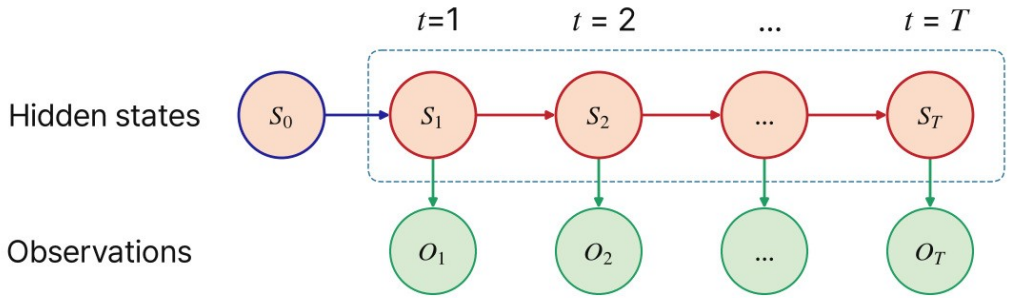
π: probability of each state at t = 1 (from a start node s0)

λ = {A, B, π}

- Red: transitions A
- Green: emissions B
- Blue: start π

- Parameters of a hidden Markov model (λ)
  - $A(a_{ij})$: State transition probability matrix
  - $B(b_{jk})$: Emission probability matrix
  - $\pi(\pi_i)$: Initial state probability matrix





$$\pi = \begin{matrix} \\ S1 \\ S2 \\ S3 \end{matrix} \begin{matrix} S_{t=0} \\ \begin{bmatrix} \pi_1 \\ \pi_2 \\ \pi_3 \end{bmatrix} \end{matrix} \qquad A = \begin{matrix} \\ S1 \\ S2 \\ S3 \end{matrix} \begin{matrix} \begin{matrix} S1 & S2 & S3 \end{matrix} \\ \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \end{matrix} \qquad B = \begin{matrix} \\ S1 \\ S2 \\ S3 \end{matrix} \begin{matrix} \begin{matrix} O1 & O2 & O3 \end{matrix} \\ \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} \end{matrix}$$

Each parameter is a probability distribution that sums to one

Definitions from the original slide (index of the B sum corrected to k)

Model $\lambda = \{A, B, \pi\}$

A: next state $a_{ij} = p(q_{t+1} = s_j \mid q_t = s_i), \quad \sum_{j=1}^{n} a_{ij} = 1$

B: what is observed $b_j(v_k) = P(o_t = v_k \mid q_t = s_j), \quad \sum_{k=1}^{m} b_j(v_k) = 1$

π: starting state $\pi_i = P(q_1 = s_i), \quad \sum_{i=1}^{n} \pi_i = 1$

Every HMM task reduces to one of three problems

Evaluation · Decoding · Learning

Three problems

- Evaluation: P(O | λ)
- Decoding: best hidden path S
- Learning: find λ* from data

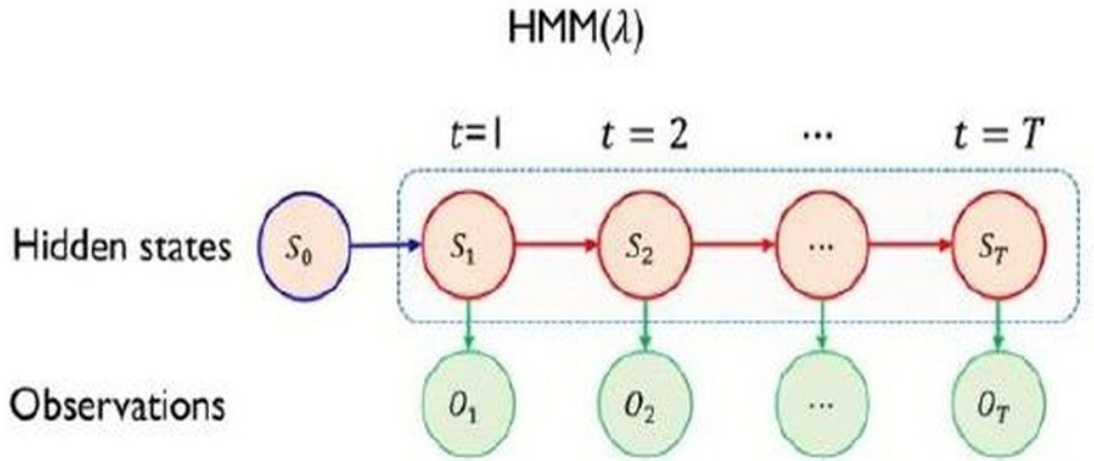
❖ Hidden Markov model $\lambda = (A, B, \pi)$

❖ Three problems of hidden Markov model

① Given HMM (λ^*) and $\mathbf{O}$, find the probability of $\mathbf{O}$ → **Evaluation** problem

② Given HMM (λ^*) and $\mathbf{O}$, find the optimal hidden state sequence ($\mathbf{S}$) → **Decoding** problem

③ Given $X = \{\mathbf{O_1}, \ldots, \mathbf{O_N}\}$, find the HMM ($\lambda^*$) → **Learning** problem

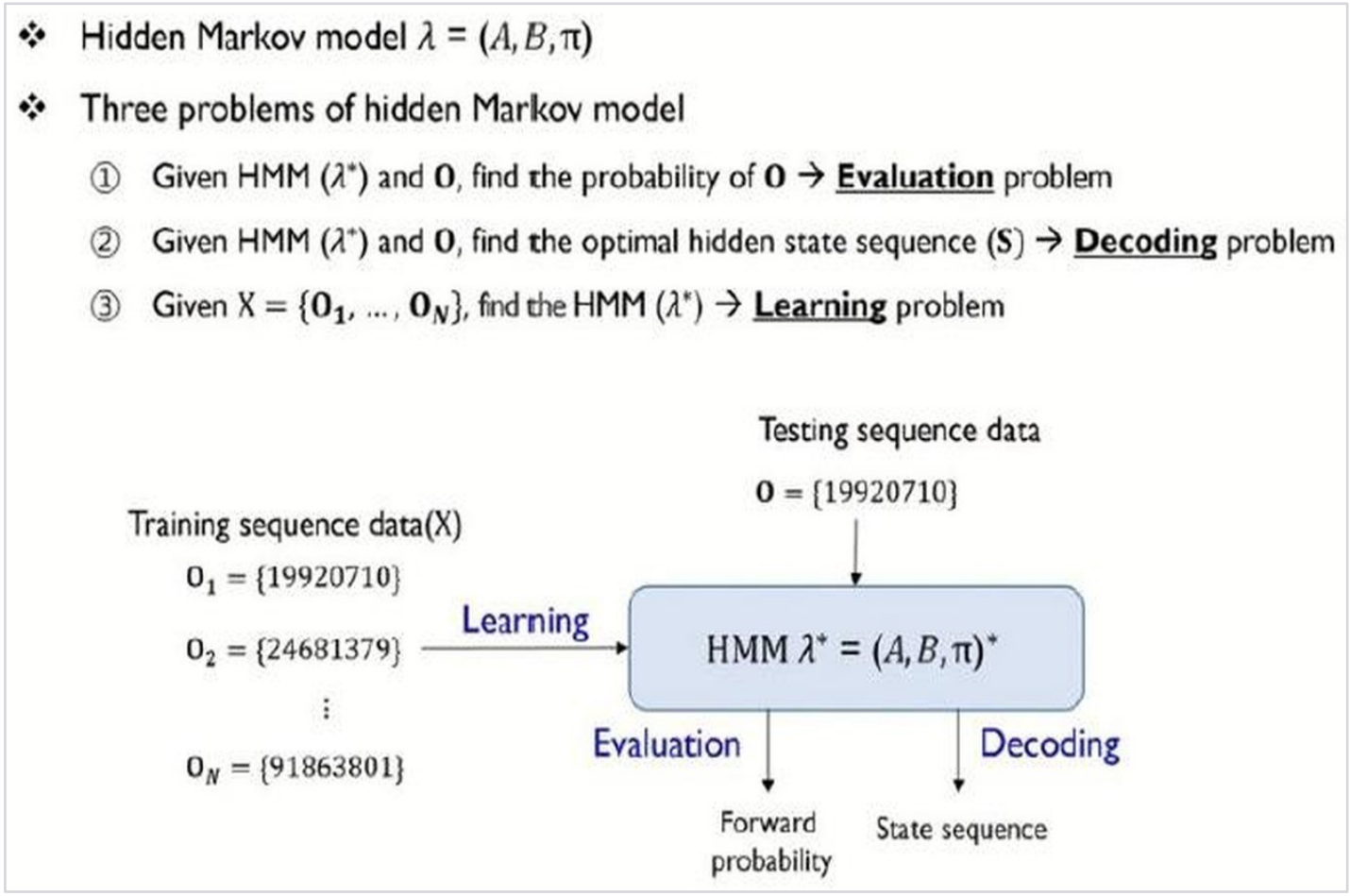


Learning builds the model that evaluation and decoding then use

Training sequences → λ*; a test sequence → probability or state path

Workflow

- Train on O1...ON
- Evaluate: forward probability
- Decode: state sequence



03

PART 3 · EVALUATION

Evaluation and the Forward Algorithm

How likely is an observed sequence?

Dr. Jung's HMM fixes every parameter of the running example

Question: P(walk, walk, research, shop)?

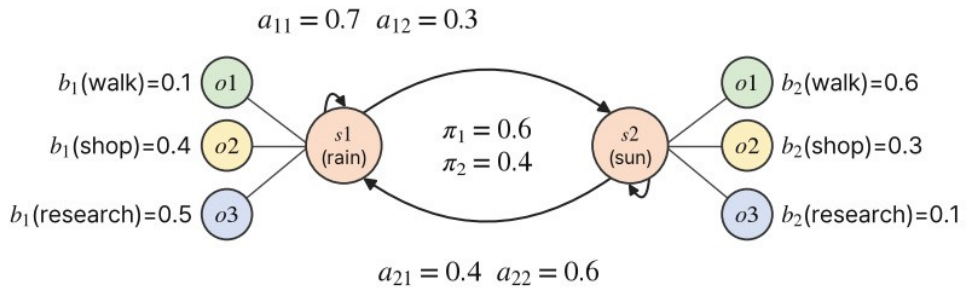
Parameters

- π: rain 0.6, sun 0.4
- A: rain→rain 0.7, sun→sun 0.6
- B(rain): walk .1 shop .4 research .5
- B(sun): walk .6 shop .3 research .1

◆ Evaluation problem

- Problem: Given HMM (λ^*) and **O**, find the probability of **O**
- Solution: Forward algorithm
- Example: How likely is Dr. Jung to walk, walk, do research, then shop on four days?

Dr. Jung's HMM



O = (o_1 = walk, o_2 = walk, o_3 = research, o_4 = shop)

Probability (O) = (o_1 = walk, o_2 = walk, o_3 = research, o_4 = shop) = ?

Brute-force enumeration grows as N to the power T

Every hidden path must be scored and summed — the original enumerates 16 weather paths

1	Rain	Rain	Rain	Rain
2	Rain	Rain	Rain	Sun
3	Rain	Rain	Sun	Rain
4	Rain	Sun	Rain	Rain
5	Sun	Rain	Rain	Rain
6	Sun	Sun	Rain	Rain
7	Rain	Sun	Sun	Rain
8	Rain	Rain	Sun	Sun
9	Sun	Rain	Rain	Sun
10	Sun	Rain	Sun	Rain
11	Rain	Sun	Rain	Sun
12	Sun	Sun	Sun	Rain
13	Rain	Sun	Sun	Sun
14	Sun	Rain	Sun	Sun
15	Sun	Sun	Rain	Sun
16	Sun	Sun	Sun	Sun

Total number of paths = 2^4 = 16

Why it fails

- 2 states, 4 days: 16 paths
- 3 states, 20 days: ≈3.5 billion
- Need a recursion

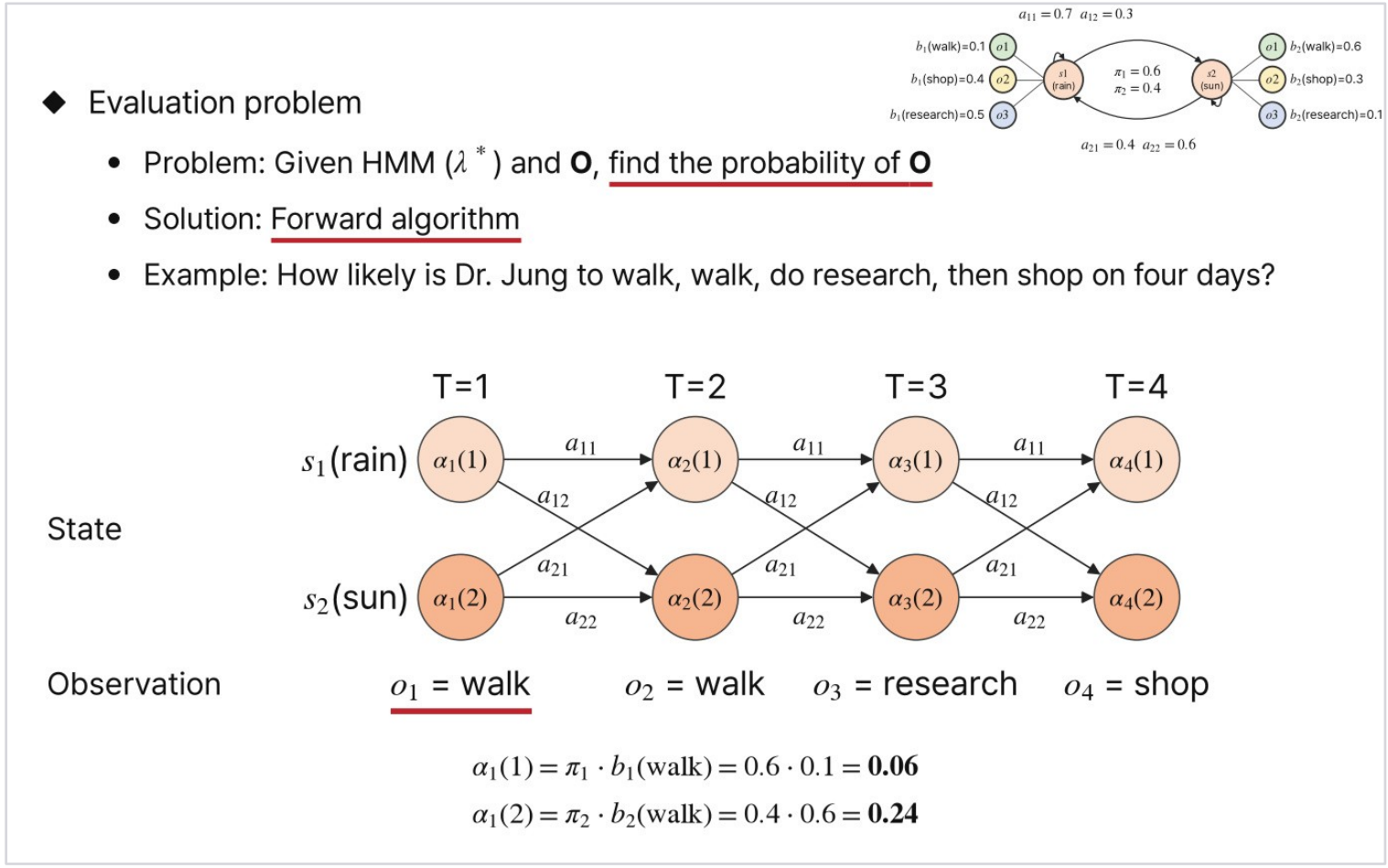
$$\text{number of hidden paths} = N^T \qquad 2^4 = 16, \quad 3^{20} = 3{,}486{,}784{,}401$$

The forward algorithm starts with prior times first emission

t = 1, observation walk

Step 1

- α1(rain) = 0.6 × 0.1 = 0.06
- α1(sun) = 0.4 × 0.6 = 0.24

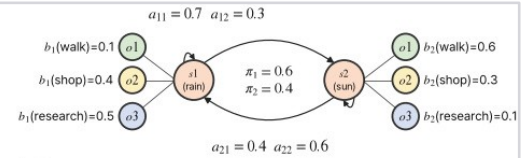


Each forward value sums all paths arriving at that state

t = 2, observation walk

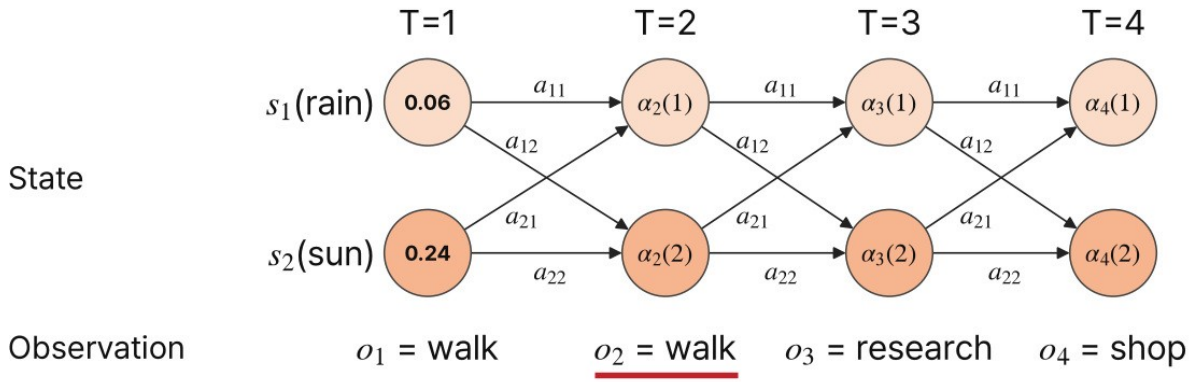
Step 2

- α2(rain) = 0.0138
- α2(sun) = 0.0972
- Sum over predecessors, then × b



◆ Evaluation problem

- Problem: Given HMM (λ^*) and **O**, find the probability of **O**
- Solution: Forward algorithm
- Example: How likely is Dr. Jung to walk, walk, do research, then shop on four days?



$$\alpha_2(1) = (\alpha_1(1) \cdot a_{11} + \alpha_1(2) \cdot a_{21}) \cdot b_1(\text{walk}) = (\mathbf{0.06} \cdot 0.7 + \mathbf{0.24} \cdot 0.4) \cdot 0.1 = \mathbf{0.0138}$$

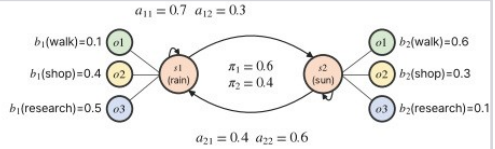
$$\alpha_2(2) = (\alpha_1(1) \cdot a_{12} + \alpha_1(2) \cdot a_{22}) \cdot b_2(\text{walk}) = (\mathbf{0.06} \cdot 0.3 + \mathbf{0.24} \cdot 0.6) \cdot 0.6 = \mathbf{0.0972}$$

The recursion reuses yesterday's forward values

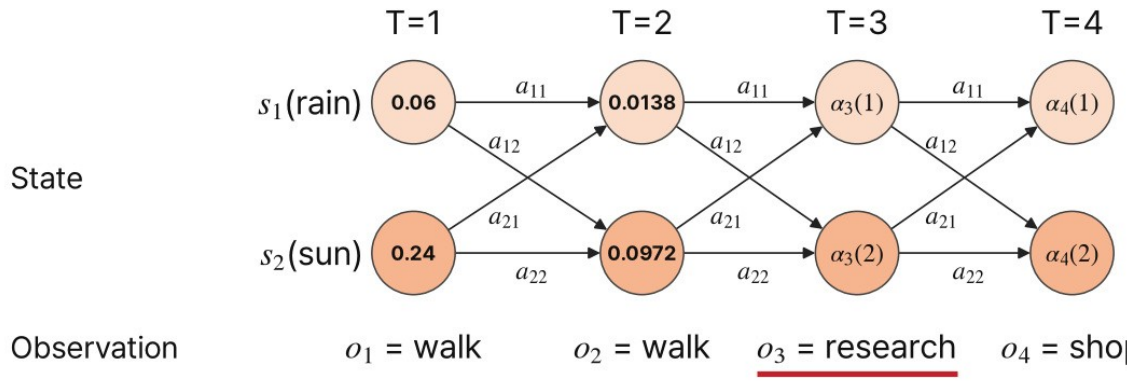
t = 3, observation research

Step 3

- α3(rain) = 0.02427
- α3(sun) = 0.00625
- Research favours rain



- Evaluation problem
  - Problem: Given HMM (λ^*) and **O**, find the probability of **O**
  - Solution: Forward algorithm
  - Example: How likely is Dr. Jung to walk, walk, do research, then shop on four days?



$$\alpha_3(1) = (\alpha_2(1) \cdot a_{11} + \alpha_2(2) \cdot a_{21}) \cdot b_1(\text{research}) = (\mathbf{0.0138} \cdot 0.7 + \mathbf{0.0972} \cdot 0.4) \cdot 0.5 = \mathbf{0.02427}$$

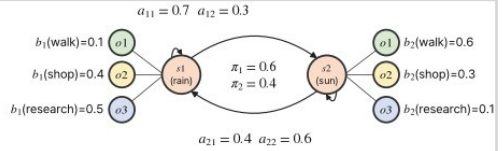
$$\alpha_3(2) = (\alpha_2(1) \cdot a_{12} + \alpha_2(2) \cdot a_{22}) \cdot b_2(\text{research}) = (\mathbf{0.0138} \cdot 0.3 + \mathbf{0.0972} \cdot 0.6) \cdot 0.1 = \mathbf{0.00625}$$

The last step produces forward values for every final state

t = 4, observation shop

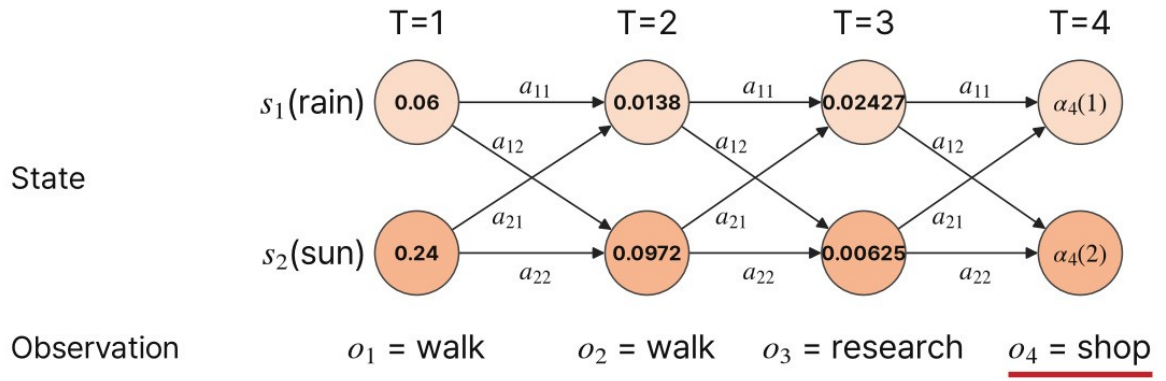
Step 4

- α4(rain) = 0.00780
- α4(sun) = 0.00331



◆ Evaluation problem

- Problem: Given HMM (λ^*) and **O**, find the probability of **O**
- Solution: Forward algorithm
- Example: How likely is Dr. Jung to walk, walk, do research, then shop on four days?



$$\alpha_4(1) = (\alpha_3(1) \cdot a_{11} + \alpha_3(2) \cdot a_{21}) \cdot b_1(\text{shop}) = (\mathbf{0.02427} \cdot 0.7 + \mathbf{0.00625} \cdot 0.4) \cdot 0.4 = \mathbf{0.00780}$$

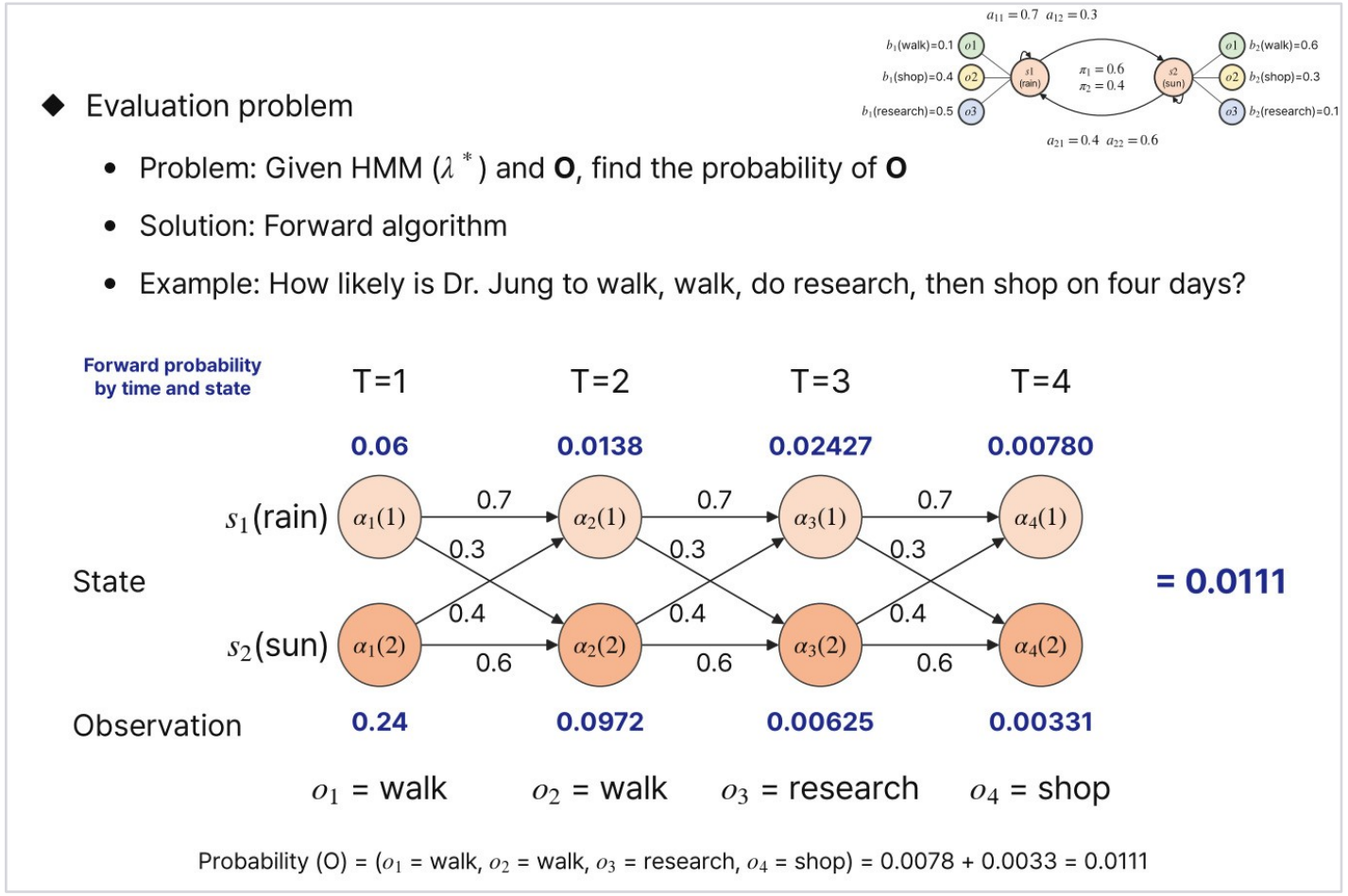
$$\alpha_4(2) = (\alpha_3(1) \cdot a_{12} + \alpha_3(2) \cdot a_{22}) \cdot b_2(\text{shop}) = (\mathbf{0.02427} \cdot 0.3 + \mathbf{0.00625} \cdot 0.6) \cdot 0.3 = \mathbf{0.00331}$$

The sequence probability equals the sum of the final forward values

P(O) = 0.0078 + 0.0033 = 0.0111

Result

- Add the last column
- P(O | λ) = 0.0111
- Cost grows as N^2T, not N^T



The forward recursion is a sum over predecessors times an emission

Forward probability α_t(i) — formula from the original slide

$$\alpha_1(i) = \pi_i \, b_i(o_1)$$

$$\alpha_t(i) = \left[\sum_{j=1}^{n} \alpha_{t-1}(j) \, a_{ji} \right] b_i(o_t), \quad 2 \le t \le T$$

$$p(O \mid \lambda) = \sum_{j=1}^{n} \alpha_T(j)$$

Use

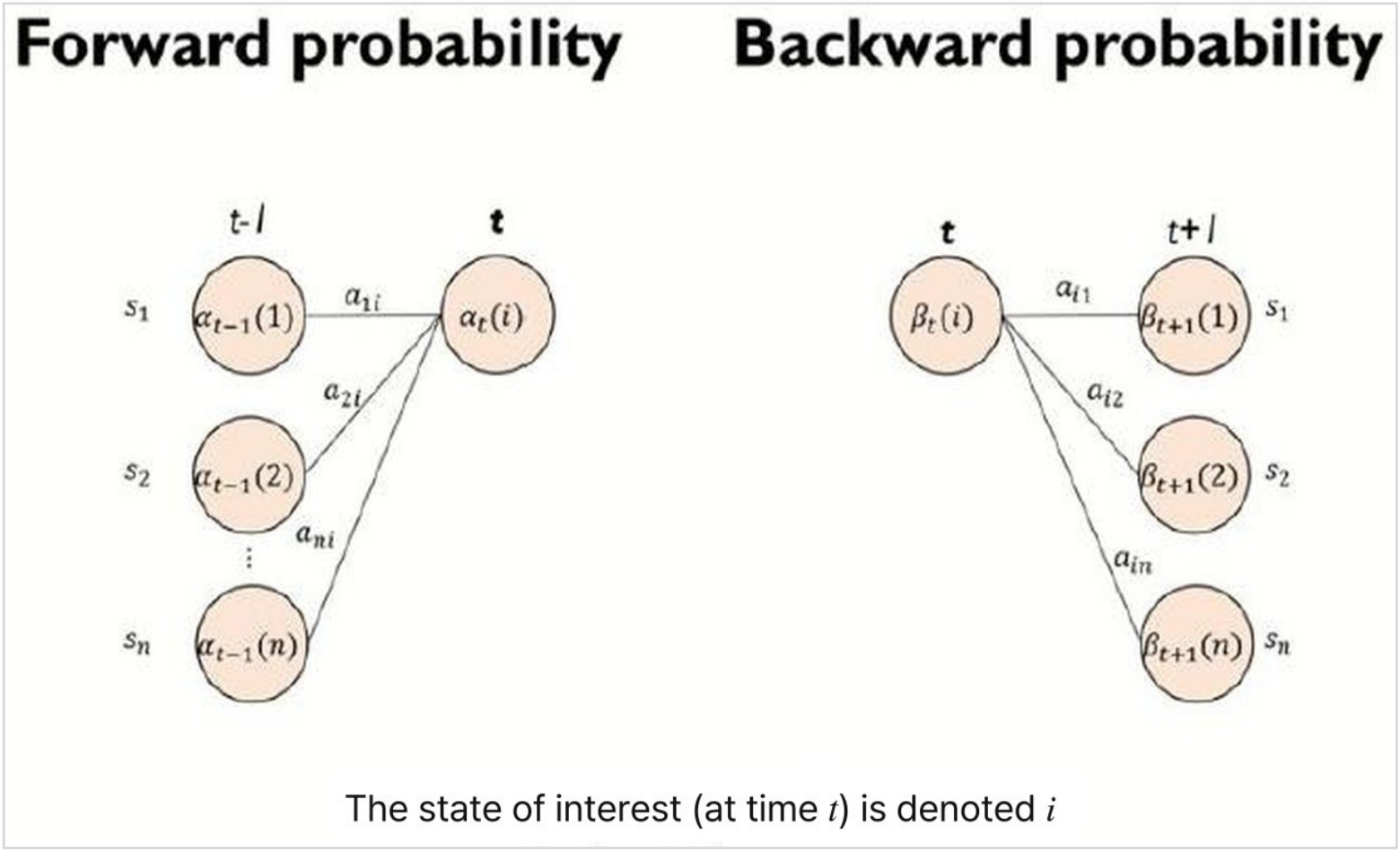
- P(O | λ) tells which model a sequence most likely belongs to (Dr. Jung vs Dr. Cho)
- → sequence classification

Backward probabilities run the same idea in reverse time

Forward: past → now · Backward: future → now

Mirror image

- α_t(i): from t−1 into i
- β_t(i): from i out to t+1
- Both aggregate all paths



The backward algorithm starts from one at the final time

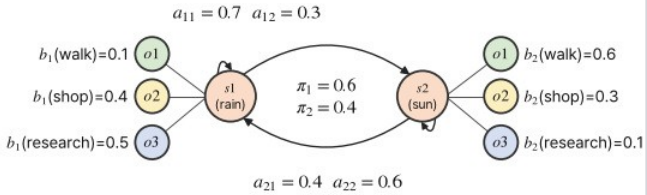
β_T(i) = 1 for every state

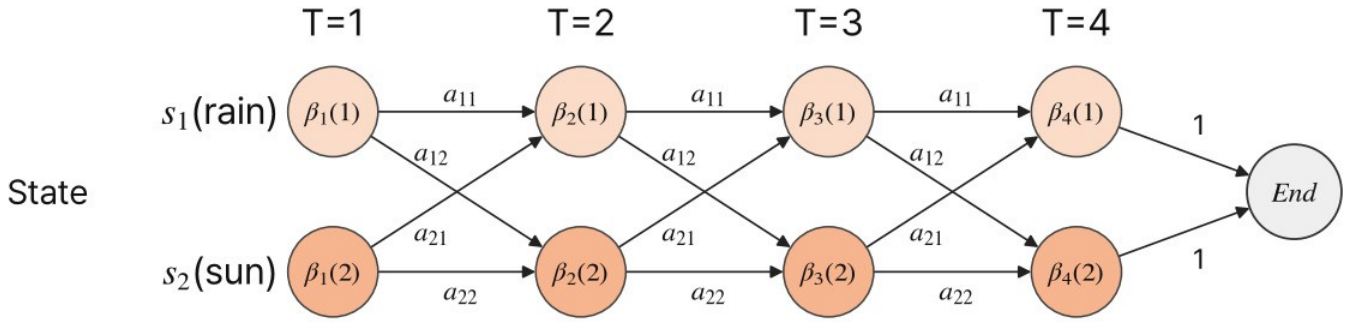
Initialise

- Nothing left to explain
- β4(rain) = β4(sun) = 1

◆ An example of **Backward** algorithm

- Problem: Given HMM (λ^*) and **O**, find the **backward** probability of **O**
- Solution: **Backward** algorithm





Observation o_1 = walk o_2 = walk o_3 = research o_4 = shop

$$\boldsymbol{\beta}_{t=T}(i) = 1$$

Each backward value weights the next day's emission and transition

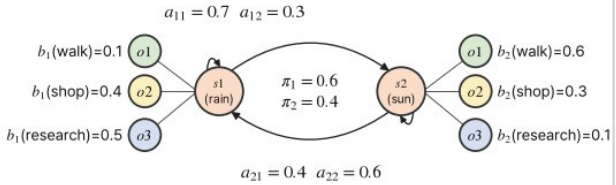
t = 3 uses o4 = shop

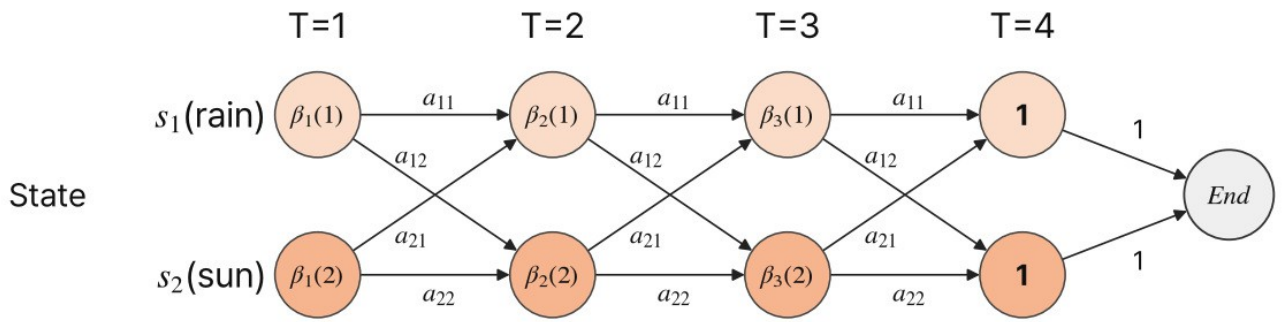
Step t = 3

- β3(rain) = 0.37
- β3(sun) = 0.34

◆ An example of **Backward** algorithm

- Problem: Given HMM (λ^*) and **O**, find the **backward** probability of **O**
- Solution: **Backward** algorithm





Observation o_1 = walk o_2 = walk o_3 = research o_4 = shop

$$\beta_3(1)=\beta_4(1)\cdot a_{11}\cdot b_1(\text{shop})+\beta_4(2)\cdot a_{12}\cdot b_2(\text{shop})=1\cdot 0.7\cdot 0.4+1\cdot 0.3\cdot 0.3=\mathbf{0.37}$$

$$\beta_3(2)=\beta_4(1)\cdot a_{21}\cdot b_1(\text{shop})+\beta_4(2)\cdot a_{22}\cdot b_2(\text{shop})=1\cdot 0.4\cdot 0.4+1\cdot 0.6\cdot 0.3=\mathbf{0.34}$$

The backward pass moves one step earlier at a time

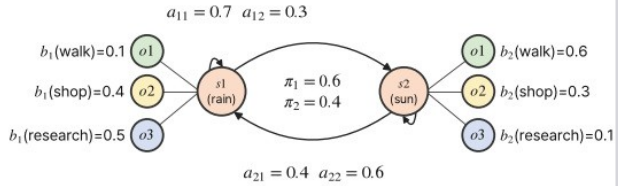
t = 2 uses o3 = research; t = 1 uses o2 = walk

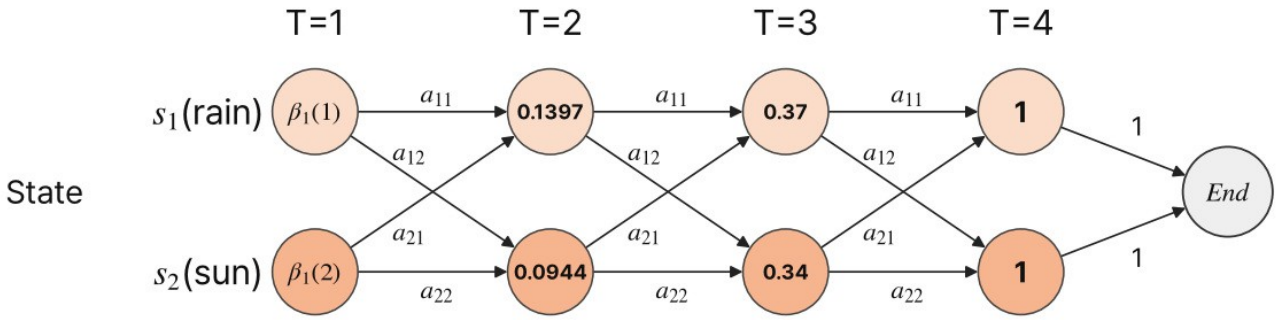
Steps t = 2, 1

- β2: 0.1397, 0.0944
- β1: 0.02677, 0.03957

◆ An example of **Backward** algorithm

- Problem: Given HMM (λ^*) and **O**, find the **backward** probability of **O**
- Solution: **Backward** algorithm





Observation o_1 = walk o_2 = walk o_3 = research o_4 = shop

$$\beta_1(1) = \beta_2(1) \cdot a_{11} \cdot b_1(\text{walk}) + \beta_2(2) \cdot a_{12} \cdot b_2(\text{walk}) = \mathbf{0.1397} \cdot 0.7 \cdot 0.1 + \mathbf{0.0944} \cdot 0.3 \cdot 0.6 = \mathbf{0.02677}$$

$$\beta_1(2) = \beta_2(1) \cdot a_{21} \cdot b_1(\text{walk}) + \beta_2(2) \cdot a_{22} \cdot b_2(\text{walk}) = \mathbf{0.1397} \cdot 0.4 \cdot 0.1 + \mathbf{0.0944} \cdot 0.6 \cdot 0.6 = \mathbf{0.03957}$$

Forward and backward passes give the same sequence probability

The original p.37 writes π1 twice; the second term uses π2

$$P(O) = \beta_1(1)\,\pi_1 b_1(o_1) + \beta_1(2)\,\pi_2 b_2(o_1) = 0.02677 \cdot 0.06 + 0.03957 \cdot 0.24 = 0.0111$$

Check

- Forward: 0.0078 + 0.0033 = 0.0111
- Backward: 0.02677·0.06 + 0.03957·0.24 = 0.0111
- Backward probabilities are used again for training (Baum-Welch)

The backward recursion is the time-reversed twin of the forward one

Formula from the original slide (time index written as t)

$$\beta_T(i) = 1$$

$$\beta_t(i) = \sum_{j=1}^{n} a_{ij}\, b_j(o_{t+1})\, \beta_{t+1}(j), \quad 1 \le t \le T-1$$

$$p(O \mid \lambda) = \sum_{i=1}^{n} \pi_i\, b_i(o_1)\, \beta_1(i)$$

Read

- Forward: previous time → next time
- Backward: next time → previous time

04

PART 4 · DECODING

Decoding and the Viterbi Algorithm

What was the weather each day?

Decoding finds the most plausible hidden state sequence

Same HMM, same observations: walk, walk, research, shop

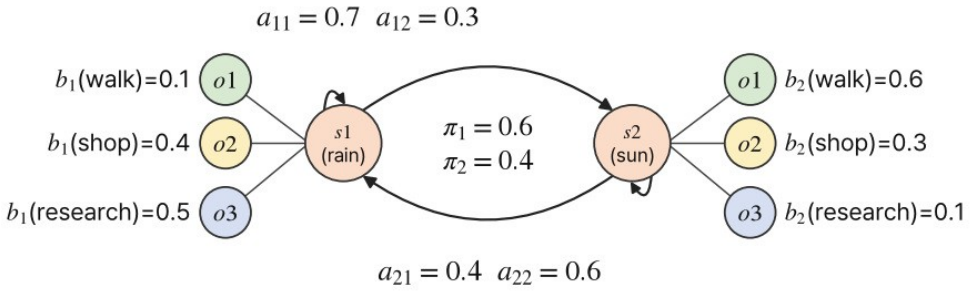
Problem

- Given λ and O
- Find optimal S
- Solution: Viterbi

◆ Decoding problem → the core of HMM!!

- Problem: Given HMM (λ^*) and **O**, find the optimal **S** (determine the most plausible hidden state sequence)
- Solution: Viterbi algorithm
- Example: If Dr. Jung walked, walked, did research, then shopped, what was each day's weather?

Dr. Jung's HMM



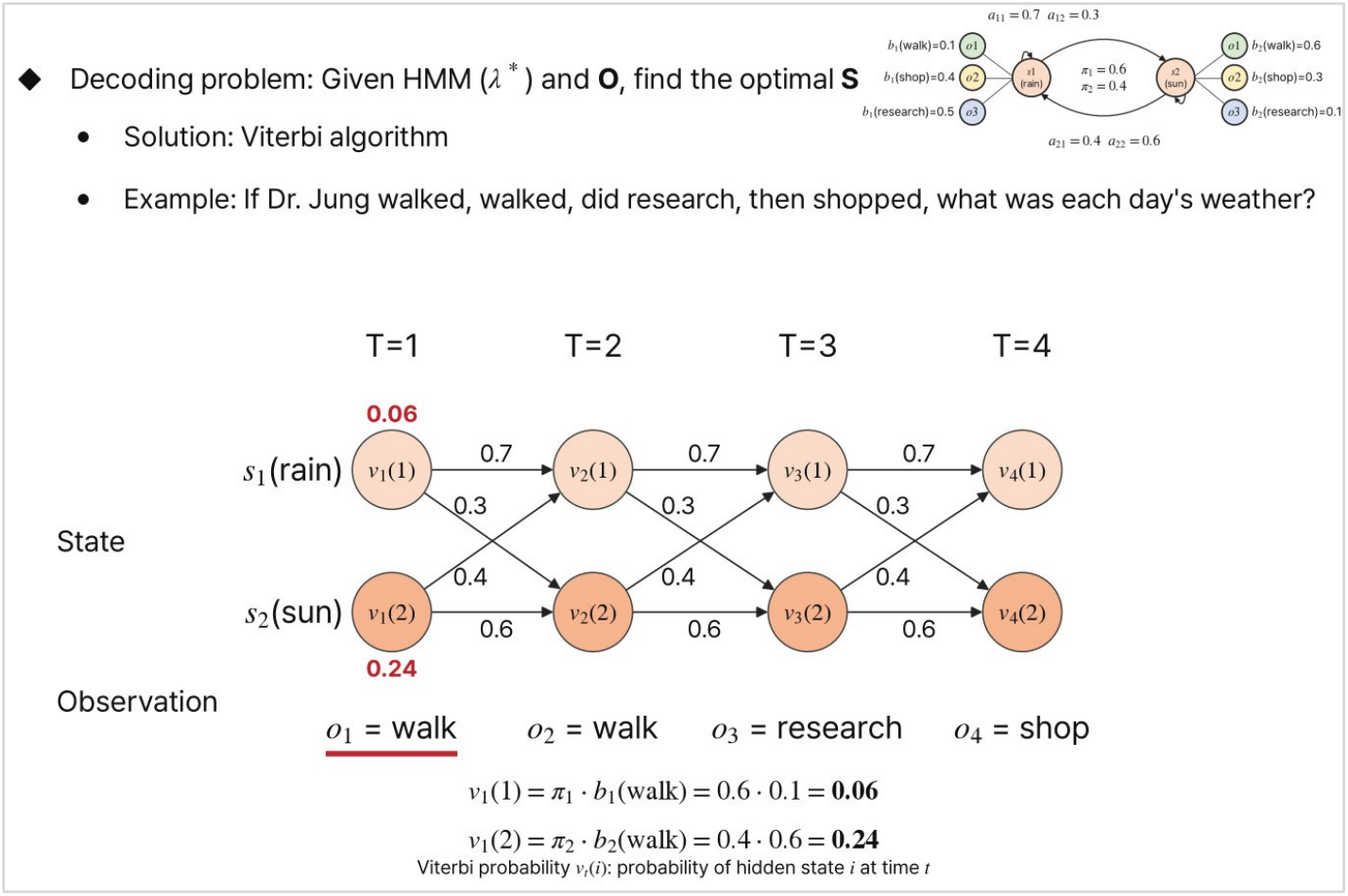
O = (o_1 = walk, o_2 = walk, o_3 = research, o_4 = shop)

Viterbi starts exactly like the forward algorithm

v1(i) = π_i · b_i(o1): the Viterbi probability of state i at time t

Step 1

- v1(rain) = 0.06
- v1(sun) = 0.24

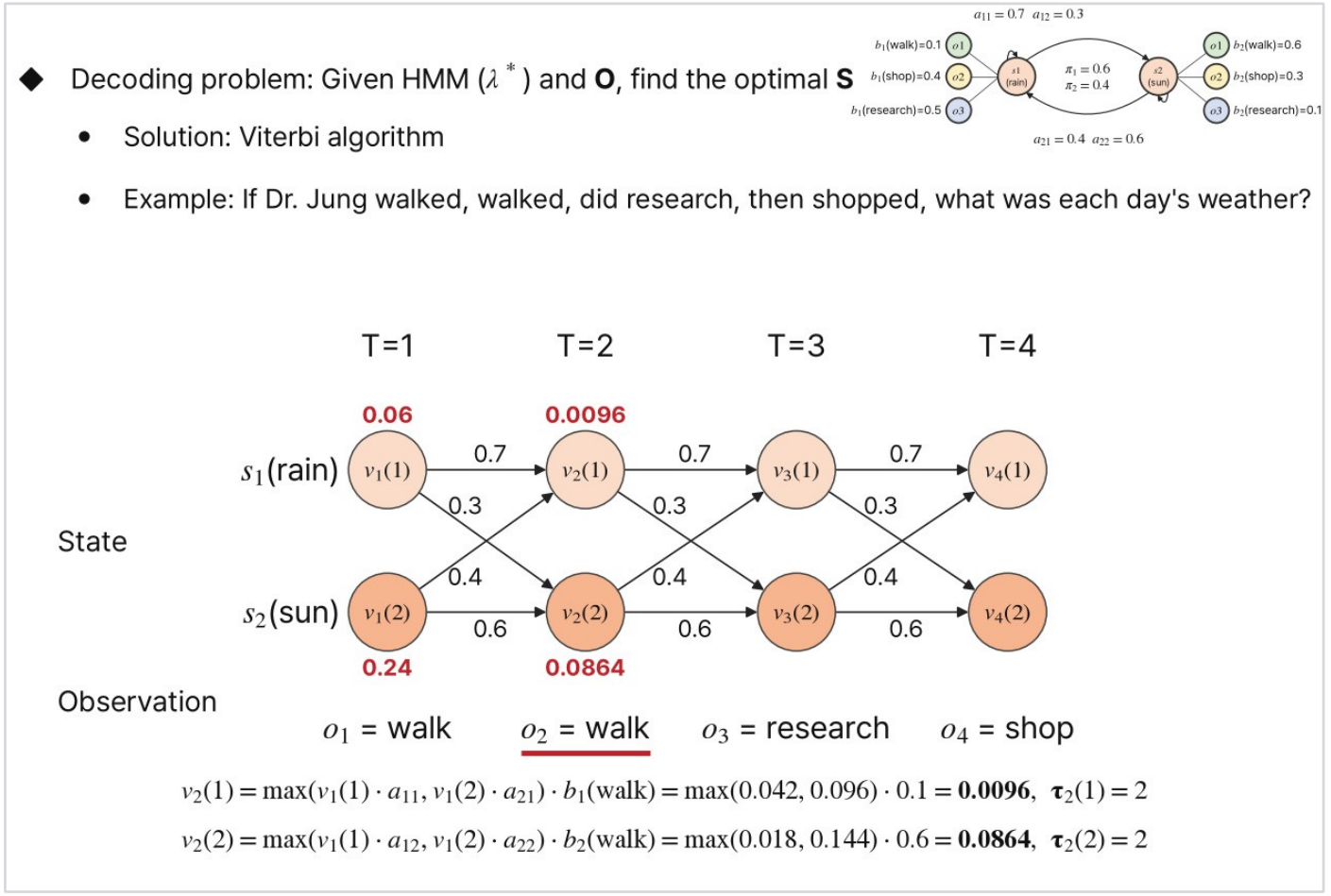


Viterbi keeps only the best incoming path and records its origin

t = 2: max replaces sum; τ stores the arg max

Step 2

- v2(rain) = 0.0096, τ = sun
- v2(sun) = 0.0864, τ = sun

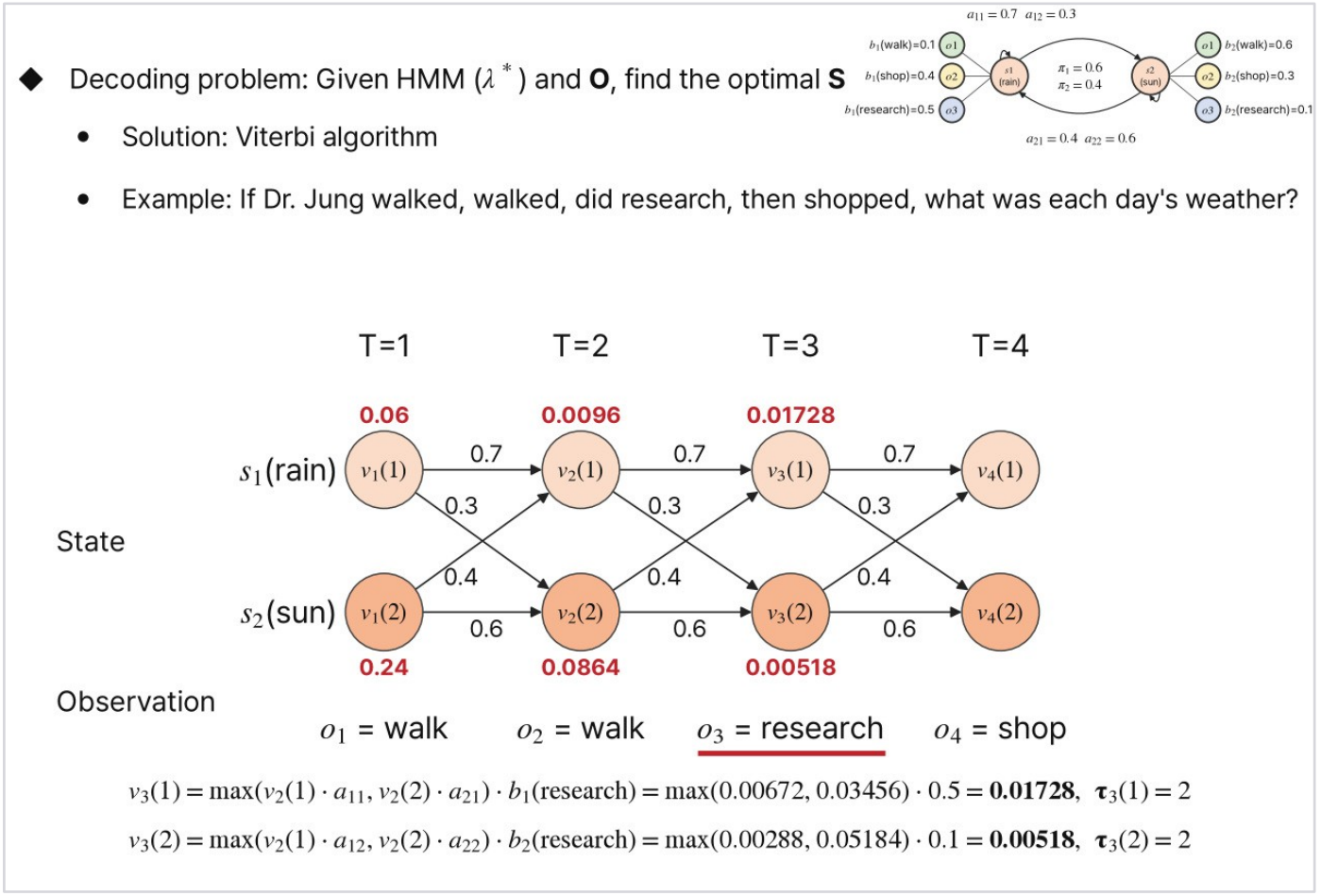


Research on day 3 switches the best state to rain

t = 3

Step 3

- v3(rain) = 0.01728, τ = sun
- v3(sun) = 0.00518, τ = sun

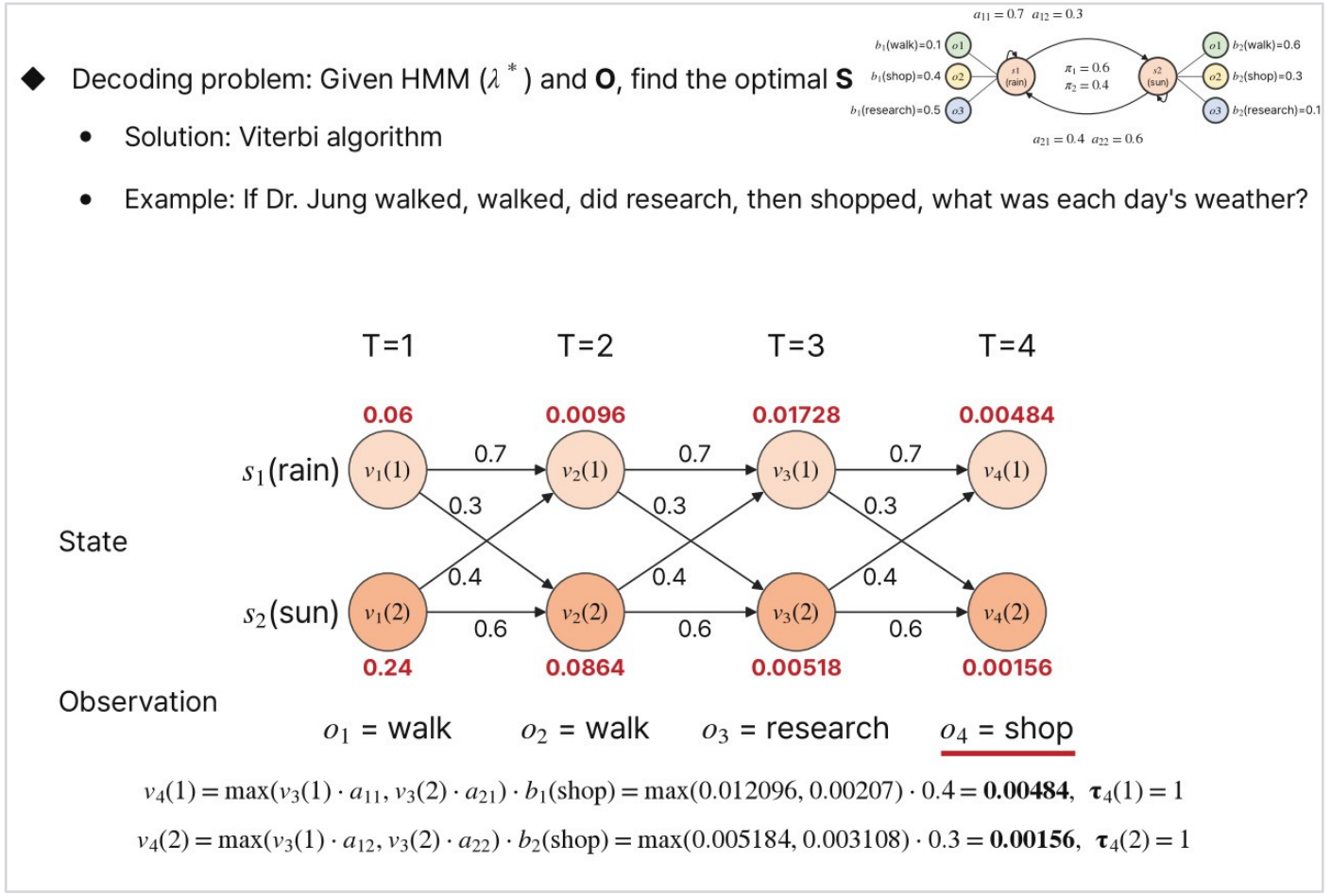


The final column holds the best path score for each end state

t = 4 (the original b(research) is corrected to b(shop))

Step 4

- v4(rain) = 0.00484, τ = rain
- v4(sun) = 0.00156, τ = rain

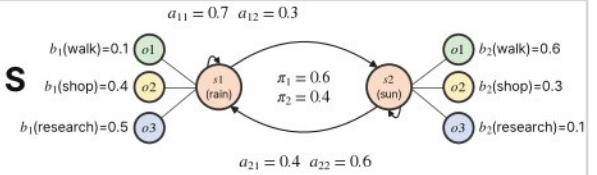


Backpointers store the best predecessor at every node

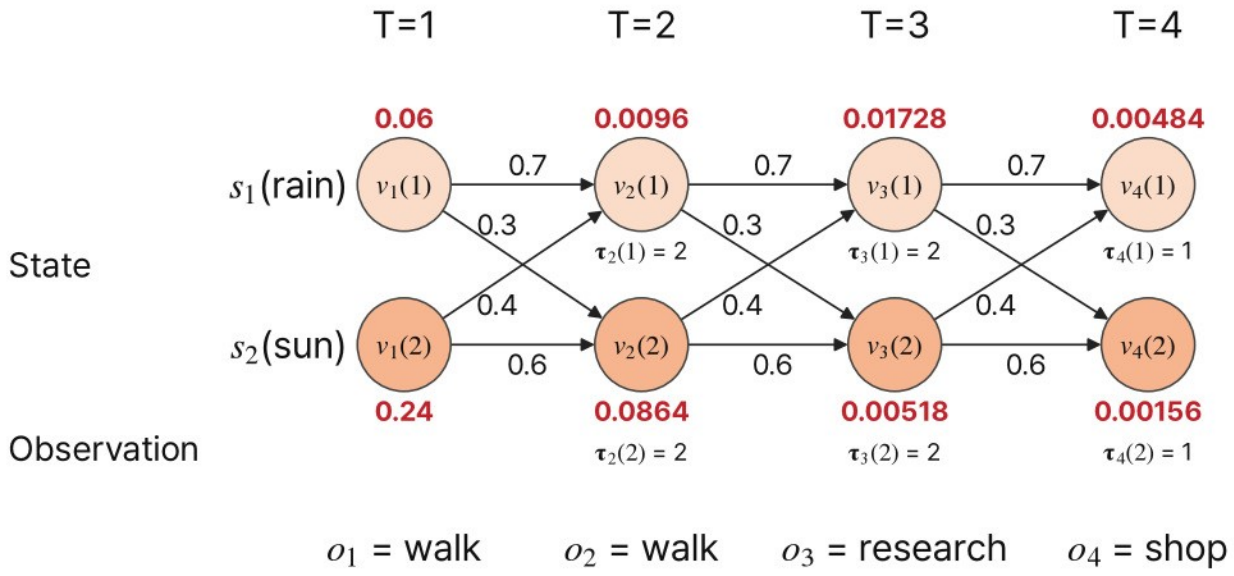
τ_t(i) is the state at t−1 on the best path into i

Backpointers

- τ2 = (sun, sun)
- τ3 = (sun, sun)
- τ4 = (rain, rain)



- Decoding problem: Given HMM (λ^*) and **O**, find the optimal **S**
  - Solution: Viterbi algorithm
  - Example: If Dr. Jung walked, walked, did research, then shopped, what was each day's weather?

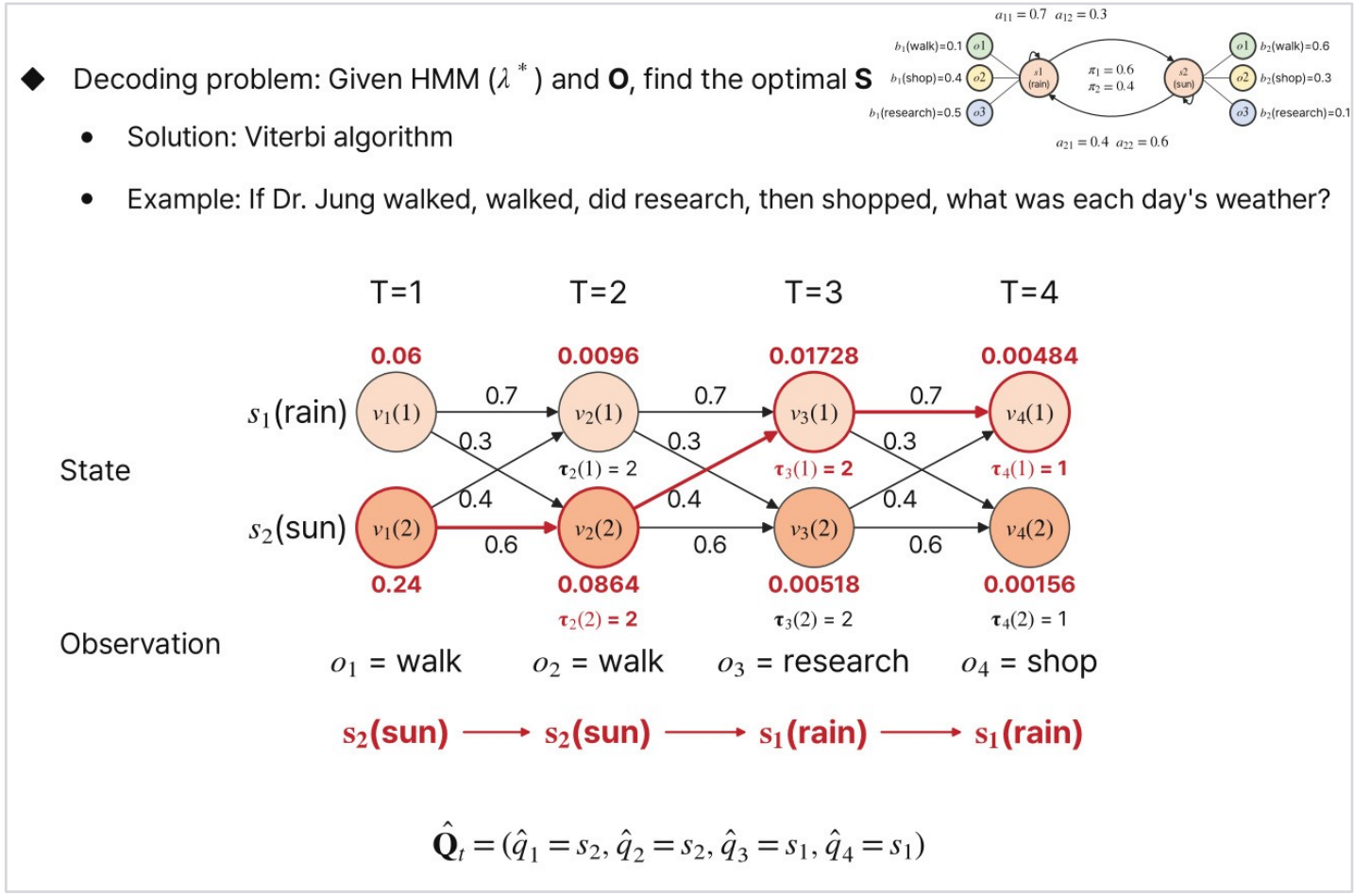


Backtracking from the best final state recovers the whole path

End at rain (0.00484) and follow τ backwards

Answer

- Sun → Sun → Rain → Rain
- $\hat{Q}$ = (s2, s2, s1, s1)



The Viterbi recursion replaces the forward sum with a maximum

Formulas from the original slide (b_i(o_t) in the recursion)

$$v_1(i) = \pi_i \, b_i(o_1)$$

$$v_t(i) = \left[\max_{1 \le j \le n} v_{t-1}(j) \, a_{ji} \right] b_i(o_t)$$

$$\tau_t(i) = \arg \max_{1 \le j \le n} v_{t-1}(j) \, a_{ji}$$

$$\hat{q}_T = \arg \max_{1 \le j \le n} v_T(j), \qquad \hat{q}_t = \tau_{t+1}(\hat{q}_{t+1})$$

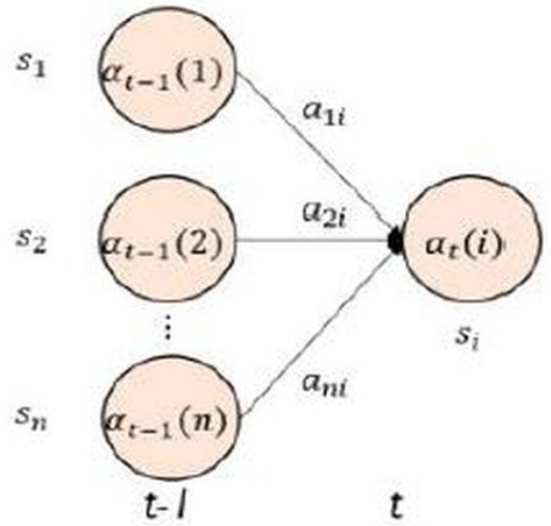
Evaluation sums over all paths while decoding takes the maximum

Same trellis, different operator (sum vs max)

One trellis

- Forward: Σ → probability of O
- Viterbi: max → best path
- Original χ corrected to v

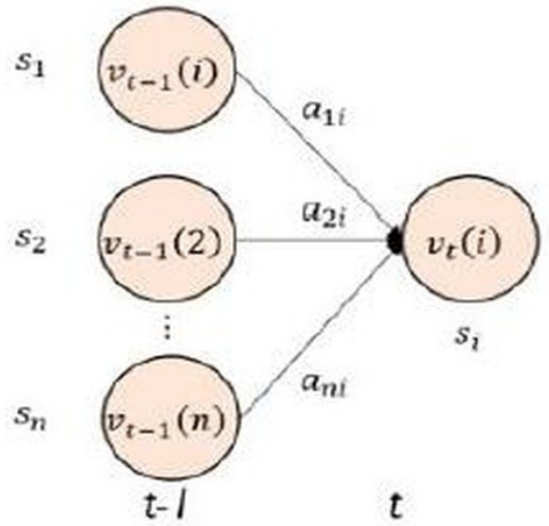
Forward algorithm for evaluation



$$\alpha_t(i) = \left[\sum_{j=1}^{n} \alpha_{t-1}(j)a_{ji}\right] \cdot b_i(o_t),$$

Sum of the probabilities of all possible paths

Viterbi algorithm for decoding



$$\mathbf{v_t(i) = \left[\max_{1 \le j \le n} v_{t-1}(j)a_{ji}\right] \cdot b_i(o_t)}$$

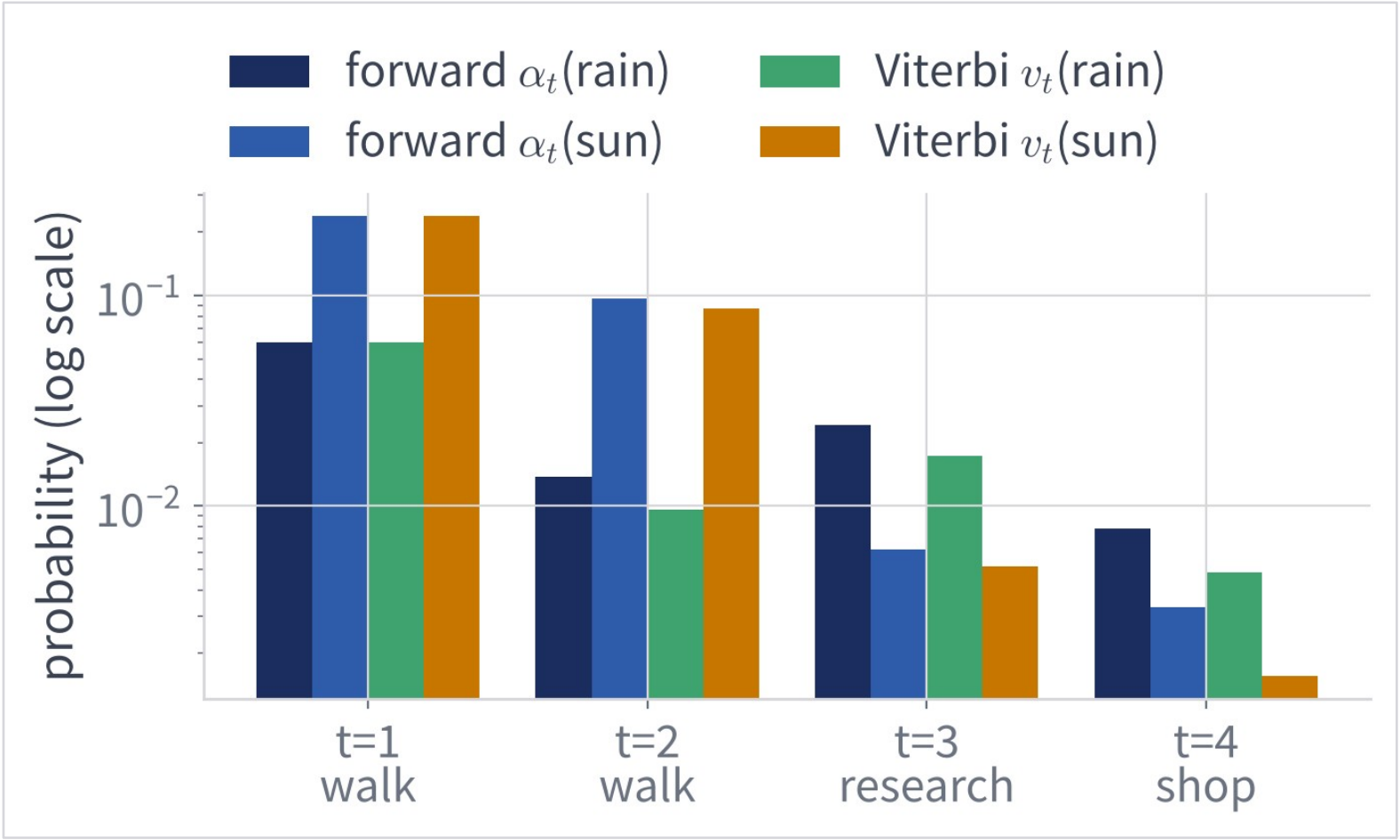
Max of the probabilities of all possible paths

Path probabilities never exceed total probabilities at any step

Values from the worked example, log scale

What to see

- Day 1: identical
- Later: max ≤ sum
- Gap = other paths



05

PART 5 · LEARNING

Parameter Learning and Baum-Welch

Finding λ* from observed sequences

Learning picks the parameters that make the data most likely

Maximum likelihood method

$$\lambda^* = \arg\max_\lambda \; P(O \mid \lambda)$$

Idea

- Given one or many observation sequences, search for the HMM that maximises P(O | λ)
- λ = {A, B, π} — π (n), A (n×n), B (n×m)

Learning improves an initial model until it is good enough

Input: HMM architecture · Output: λ* = {A*, B*, π*}

Procedure

- 1 Initialise HMM
- 2 Find λ_new with higher P(O|λ)
- 3 Stop or repeat 2

❖ Input: HMM(λ) architecture

❖ Output: HMM(λ^*)=$\{\mathbf{A}^*, \mathbf{B}^*, \boldsymbol{\pi}^*\}$ $\pi = \begin{bmatrix}\pi_1\\ \pi_2\\ \pi_3\end{bmatrix}, A = \begin{bmatrix}a_{11} & a_{12} & a_{13}\\ a_{21} & a_{22} & a_{23}\\ a_{31} & a_{32} & a_{33}\end{bmatrix}, B = \begin{bmatrix}b_{11} & b_{12} & b_{13}\\ b_{21} & b_{22} & b_{23}\\ b_{31} & b_{32} & b_{33}\end{bmatrix}$

❖ Algorithm

① Initialize HMM

② Find P(O|HMM(λ^{new})) > P(O|HMM(λ)) by a suitable method

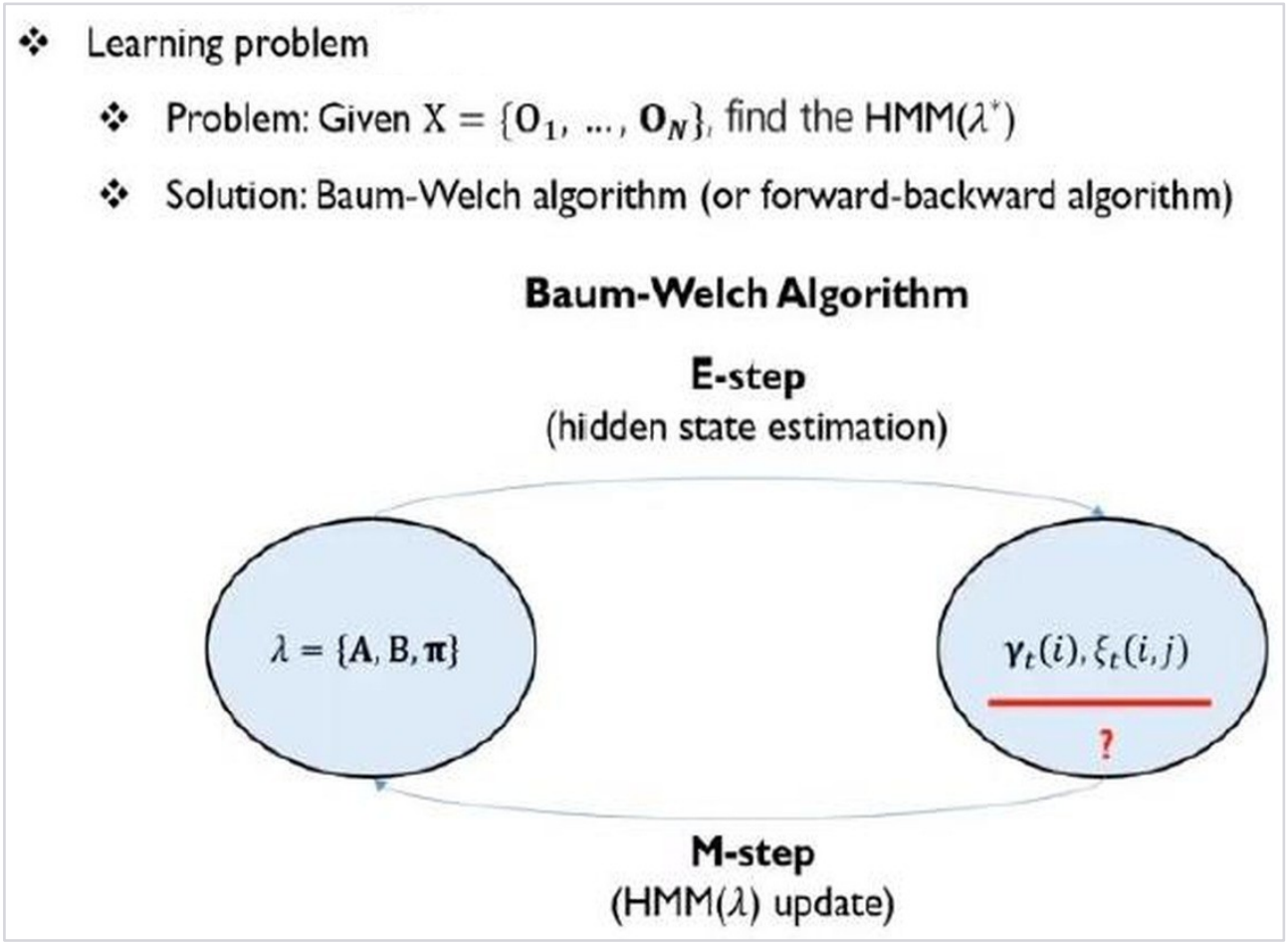
③ If satisfied, set $\hat{\lambda}$ = HMM(λ^{new}) and stop; otherwise repeat ②

Baum-Welch alternates hidden-state estimation and parameter updates

An EM algorithm also called the forward-backward algorithm

Two steps

- E-step: γ, ξ from α, β
- M-step: update λ
- Repeat until convergence

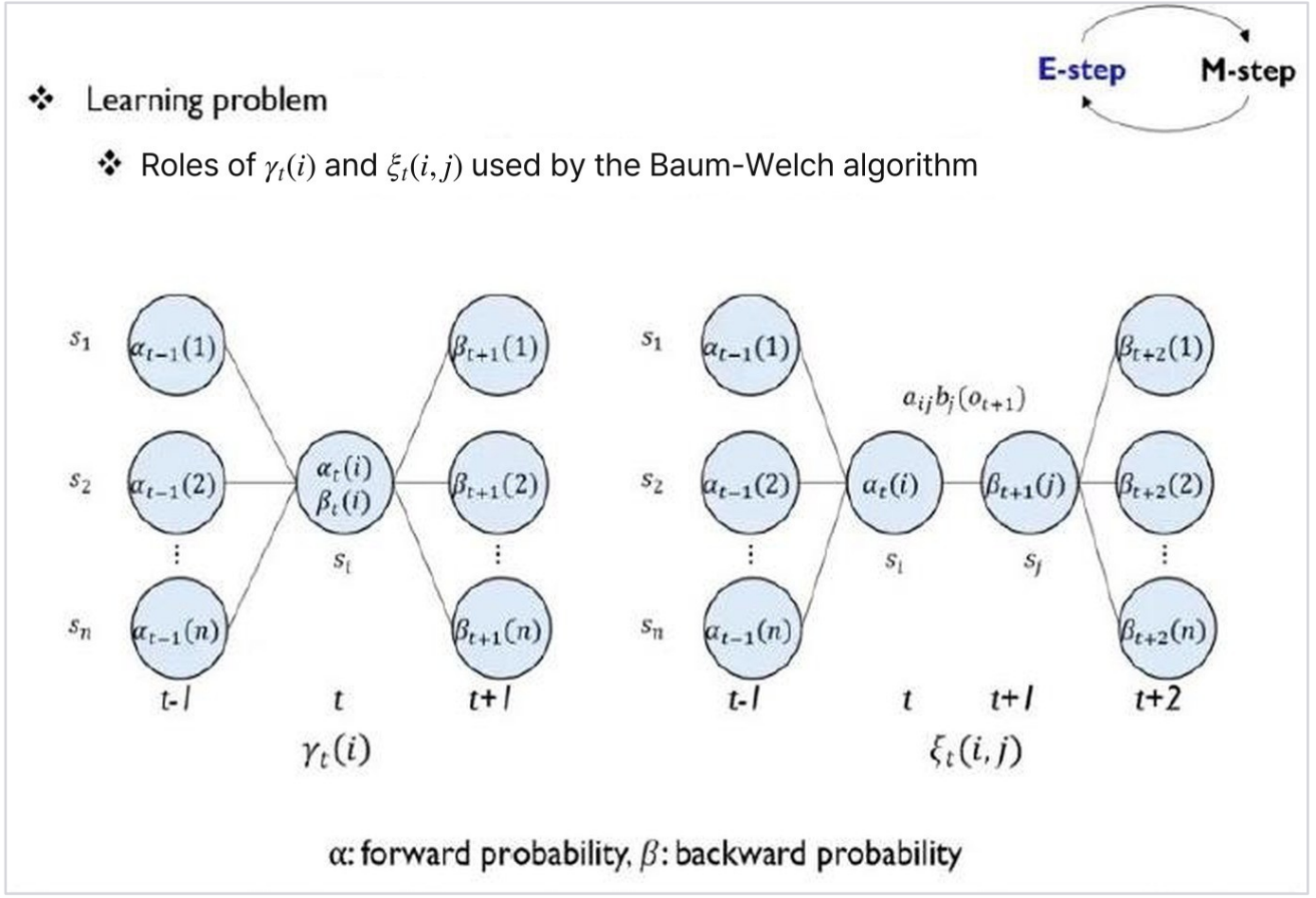


γ and ξ combine forward and backward probabilities around one time

γ_t(i): one state at t · ξ_t(i,j): a transition from t to t+1

Two quantities

- γ: in state i at t
- ξ: i at t, j at t+1
- α = left, β = right



γ normalises the probability of all paths passing through state i

α_t(i)β_t(i) = probability of all paths through state i at time t

$$\gamma_t(i) = p(q_t = s_i \mid O, \lambda) = \frac{\alpha_t(i)\,\beta_t(i)}{\sum_{j=1}^{n} \alpha_t(j)\,\beta_t(j)}$$

Read

- Numerator: paths through s_i at t · Denominator: same quantity summed over all states

ξ links state i at t to state j at t+1 through a·b

The middle term a_ij b_j(o_{t+1}) connects the two states

$$\xi_t(i,j) = \frac{\alpha_t(i)\, a_{ij}\, b_j(o_{t+1})\, \beta_{t+1}(j)}{\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_t(i)\, a_{ij}\, b_j(o_{t+1})\, \beta_{t+1}(j)}$$

Read

- E-step = compute α, β, then γ_t(i) and ξ_t(i,j)
- M-step uses them to build λ_new with P(O | λ_new) > P(O | λ)

Each new parameter is a ratio of expected counts

Expected transitions over expected visits; expected emissions over expected visits

$$\pi_i^{new} = \gamma_1(i)$$

$$a_{ij}^{new} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

$$b_i(v_k)^{new} = \frac{\sum_{t=1,\, o_t=v_k}^{T} \gamma_t(i)}{\sum_{t=1}^{T} \gamma_t(i)}$$

In words

- π: share in state i at t = 1
- a_ij: i→j moves ÷ moves out of i
- b_i(v_k): visits to i showing v_k ÷ visits to i

Training stops when the likelihood gain falls below a threshold

Or when the likelihood itself exceeds a target level

Stop rules

- a. gain < r → stop
- b. P(O|λ_new) > r → stop
- Else repeat step 2

❖ Procedure of HMM Learning

- ❖ Input: HMM(λ) architecture
- ❖ Output: HMM(λ^*)={$\mathbf{A}^*, \mathbf{B}^*, \boldsymbol{\pi}^*$}
- ❖ Algorithm

① Initialize HMM

② Find P(O|HMM(λ^{new})) > P(O|HMM(λ)) by a suitable method

③ If satisfied, set $\hat{\lambda}$ = HMM(λ^{new}) and stop; otherwise repeat ②

What does "satisfied" mean?

a. The gain is at or below a threshold (r)
   - If P(O|HMM(λ^{new})) - P(O|HMM(λ^{old})) is less than r, Stop

b. The likelihood is at or above a threshold (r)
   - If P(O|HMM(λ^{new})) is greater than r, Stop

The whole algorithm fits in a short pseudocode loop

FORWARD-BACKWARD(observations, vocabulary, hidden state set)

Loop

- Initialise A, B
- E-step: γ, ξ
- M-step: â, b̂
- Until convergence

❖ Pseudocode for Baum-Welch algorithm

function FORWARD-BACKWARD(*observations* of len T, *output vocabulary* V, *hidden state set* Q) **returns** $HMM=(A,B)$

initialize A and B

iterate until convergence

E-step

$$\gamma_t(j) = \frac{\alpha_t(j)\beta_t(j)}{\alpha_T(q_F)} \quad \forall\, t \text{ and } j$$

$$\xi_t(i,j) = \frac{\alpha_t(i)a_{ij}b_j(o_{t+1})\beta_{t+1}(j)}{\alpha_T(q_F)} \quad \forall\, t,\ i, \text{ and } j$$

M-step

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1}\sum_{k=1}^{N} \xi_t(i,k)}$$

$$\hat{b}_j(v_k) = \frac{\sum_{t=1 s.t.\ O_t=v_k}^{T} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}$$

return A, B

λ = (A, B, π) and three problems summarise the HMM

Evaluation → Forward · Decoding → Viterbi · Learning → Baum-Welch

Recap

- Evaluation: P(O|λ)
- Decoding: best S
- Learning: λ*

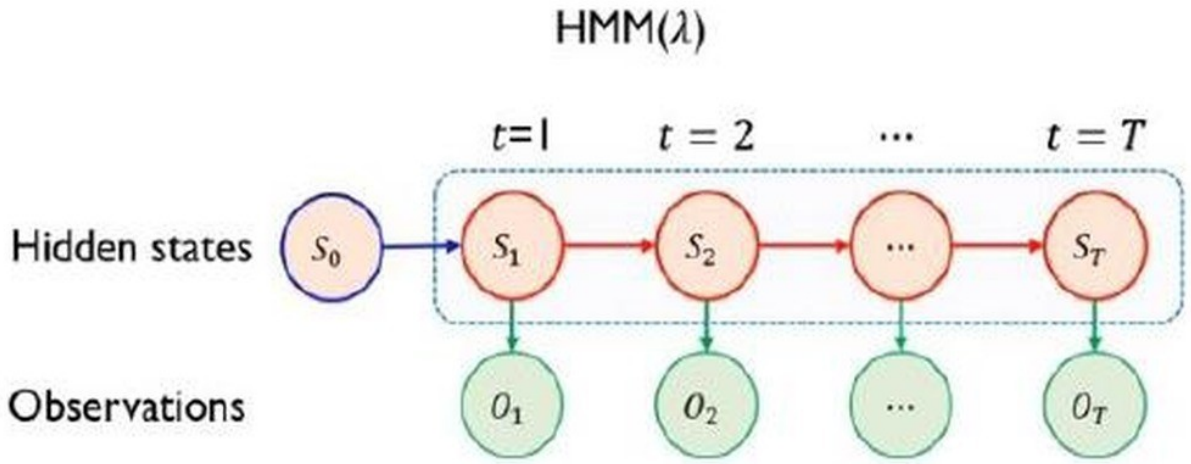
❖ Summary of hidden Markov model $\lambda = (A, B, \pi)$

❖ Three problems of hidden Markov model

① Given HMM (λ^*) and $\mathbf{O}$, find the probability of $\mathbf{O}$ → **Evaluation** problem

② Given HMM (λ^*) and $\mathbf{O}$, find the optimal $\mathbf{S}$ → **Decoding** problem

③ Given $X = \{\mathbf{O_1}, \ldots, \mathbf{O_N}\}$, find the HMM ($\lambda^*$) → **Learning** problem



06

PART 6 · APPLICATIONS

Applications

Classification and state reading in practice

Evaluation classifies a new production route as normal or defective

Observations: equipment IDs · hidden states: process IDs

How

- Train HMM(normal), HMM(defect)
- Score product 3001 on each
- Pick arg max P(O| HMM)

◆ HMM Evaluation Application

- ▸ Observation sequence: equipment ID data
- ▸ Hidden sequence: process ID data

Equipment Sequence Data in Manufacturing Processes

Time

	Proc 1	Proc 2	Proc 3	Proc 4	Proc 5	Proc 6	Proc 7	Proc 8	Proc 9	Proc 10	Quality
1	Equip 1	Equip 2	Equip 1	Equip 1	Equip 2	Equip 1	Equip 4	Equip 2	Equip 2	Equip 1	Normal
2	Equip 2	Equip 4	Equip 2	Equip 2	Equip 4	Equip 2	Equip 4	Equip 2	Equip 4	Equip 2	Normal
...	...	...	...	...	...	...	...	...	...	...	...
2999	Equip 4	Equip 1	Equip 2	Equip 4	Equip 6	Equip 5	Equip 5	Equip 1	Equip 2	Equip 3	Defect
3000	Equip 5	Equip 5	Equip 1	Equip 3	Equip 1	Equip 5	Equip 1	Equip 3	Equip 1	Equip 1	Defect

→HMM(Normal)

→HMM(Defect)

Argmax $P(O_{3001}|HMM(class))$ ... class={Normal or Defect}
class

3001	Equip 5	Equip 5	Equip 1	Equip 3	Equip 4	Equip 5	Equip 1	Equip 3	Equip 1	Equip 1	?

Decoding labels sleep stages from EEG and EMG signals

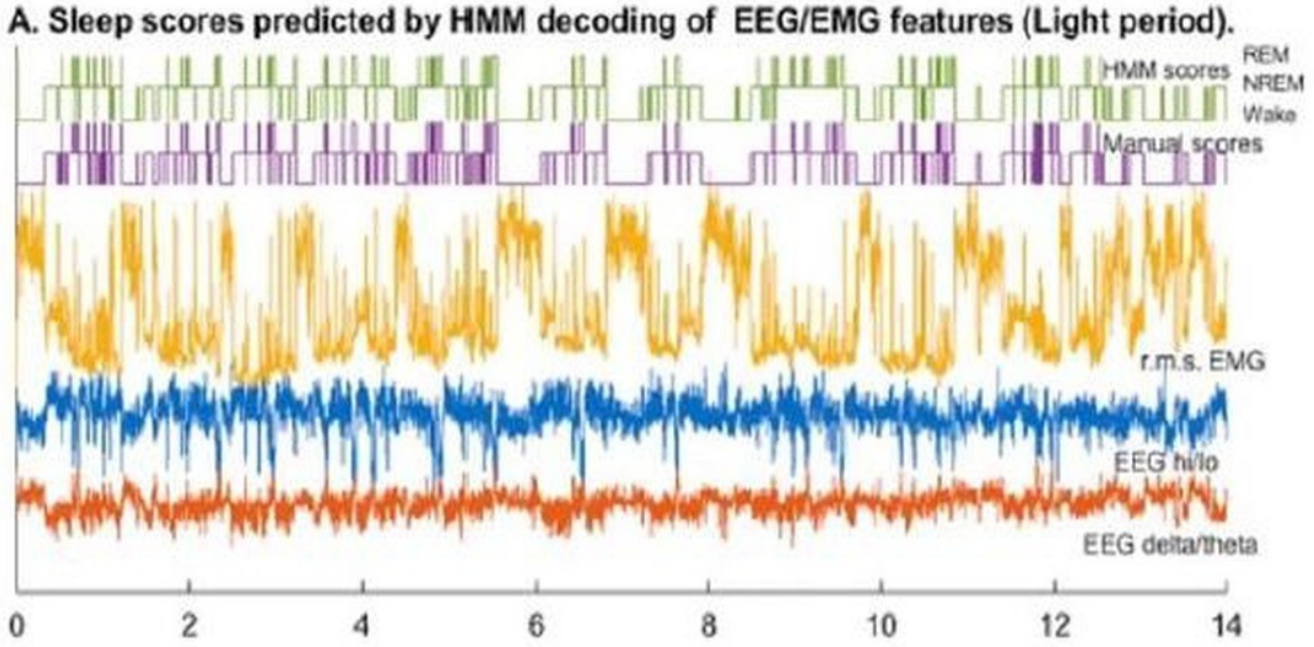
Hidden states: REM, NREM, Wake — HMM scores track manual scores

How

- Observations: EEG/EMG features
- Viterbi: stage per epoch
- Compare with manual scoring

❖ HMM Decoding Application

- ➢ Observation sequence: sleep EEG data
- ➢ Hidden sequence: REM sleep, NREM sleep, Wake



Typos in the original figures are corrected in this deck

All numbers were re-computed; the figures now show the corrected values

Original page	As printed	Now reads
p.14, 16, 56–58	B entries a22, a32	b22, b32
p.17	Σ over j for b_j(v_k)	Σ over k
p.27 (not in deck)	P(O) = 0.00111	0.0111
p.37	β1(2)·π1·b2(walk)	β1(2)·π2·b2(walk)
p.47	b(research) at t = 4	b(shop)
p.54–55	b_i(o1), χ_t−1(j)	b_i(o_t), v_t−1(j)
p.12–14, 16	a_ij labels did not match a_ij = i→j	Labels follow a_ij = i→j
p.13, 14, 16	B rows = observations (b_jk, k = state)	B rows = states (b_jk = P(o_k | s_j))
p.14, 16	Emisstion	Emission
p.22	Row 12 repeats row 7	Row 12 = sun, sun, sun, rain
p.58	P(HMM(λ_new))	P(O|HMM(λ_new))

Four questions test whether the three problems are clear

Answer with the Dr. Jung example in mind

Q1	Why does the forward algorithm cost N^2T while brute force costs N^T?	**Evaluation**
Q2	Could the most likely weather on each single day differ from the Viterbi path?	**Decoding**
Q3	If you see three research days in a row, which parameter drives the decoded weather most?	**Parameters**
Q4	Baum-Welch only guarantees a local optimum. How would you choose the initial λ?	**Learning**

One trellis answers all three HMM questions

Sum, maximise, re-estimate

1

Evaluation sums

Forward or backward recursion gives $P(O|\lambda) = 0.0111$ in the example.

2

Decoding maximises

Viterbi with backpointers gives Sun → Sun → Rain → Rain.

3

Learning re-estimates

Baum-Welch turns α, β into γ, ξ and updates A, B, π until convergence.

Next: applying HMMs to regime detection in financial time series.